

Introduction to High-Performance Machine Learning

Lecture 7

Dr. Kaoutar El Maghraoui

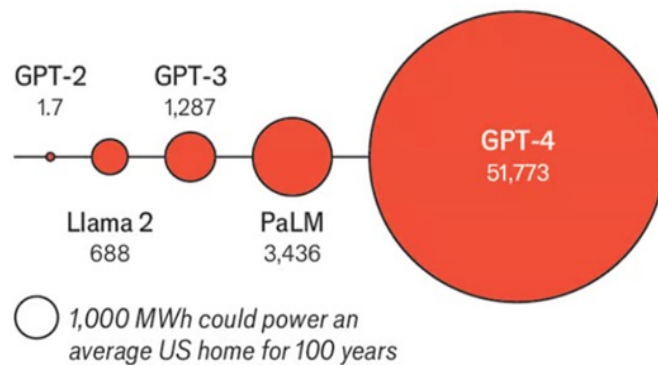
April 6, 2026



Neural Network Quantization

Increasing computational demand for Gen AI

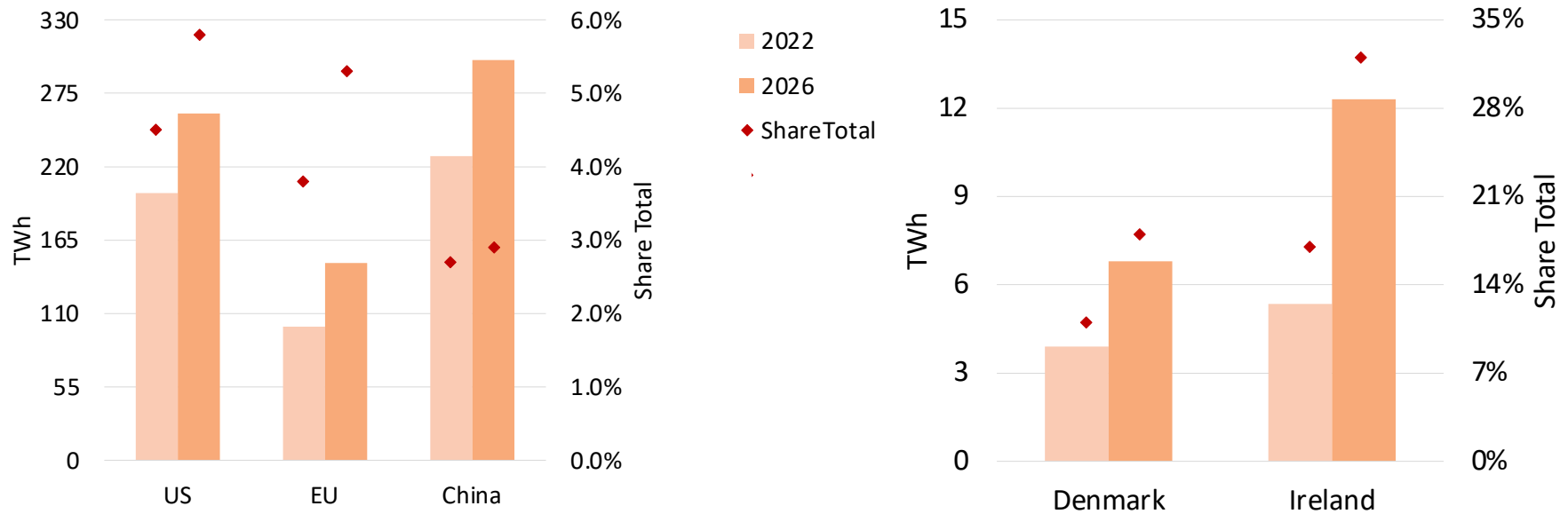
Energy used to train models, MWh



Training compute (FLOP)

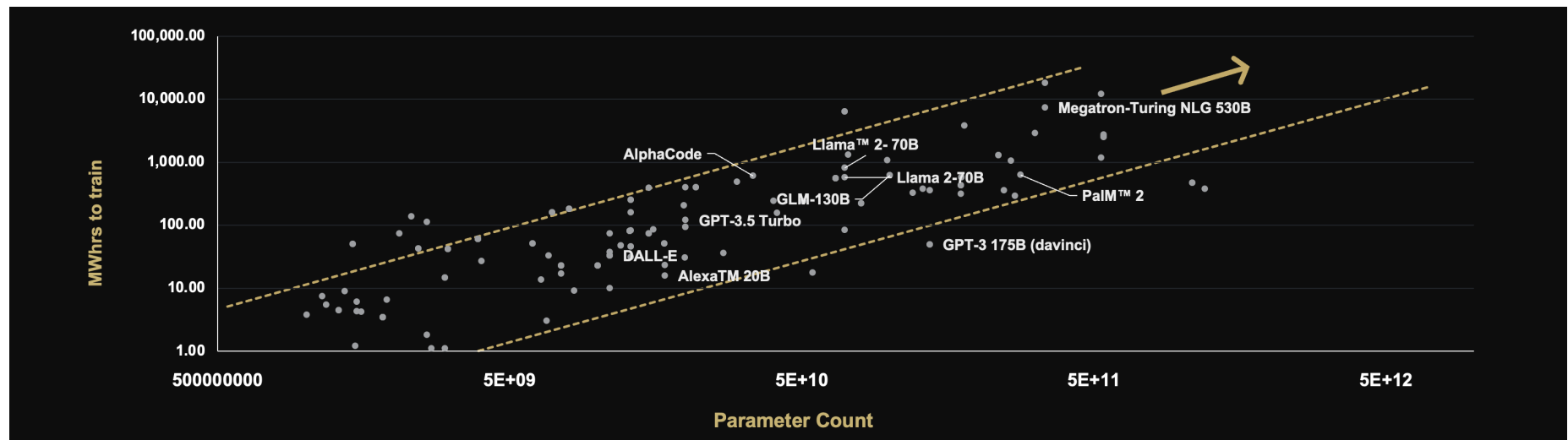


Data Center Energy Consumption



Data source: IEA (2024), Electricity 2024, IEA, Paris <https://www.iea.org/reports/electricity-2024>, License: CC BY 4.0

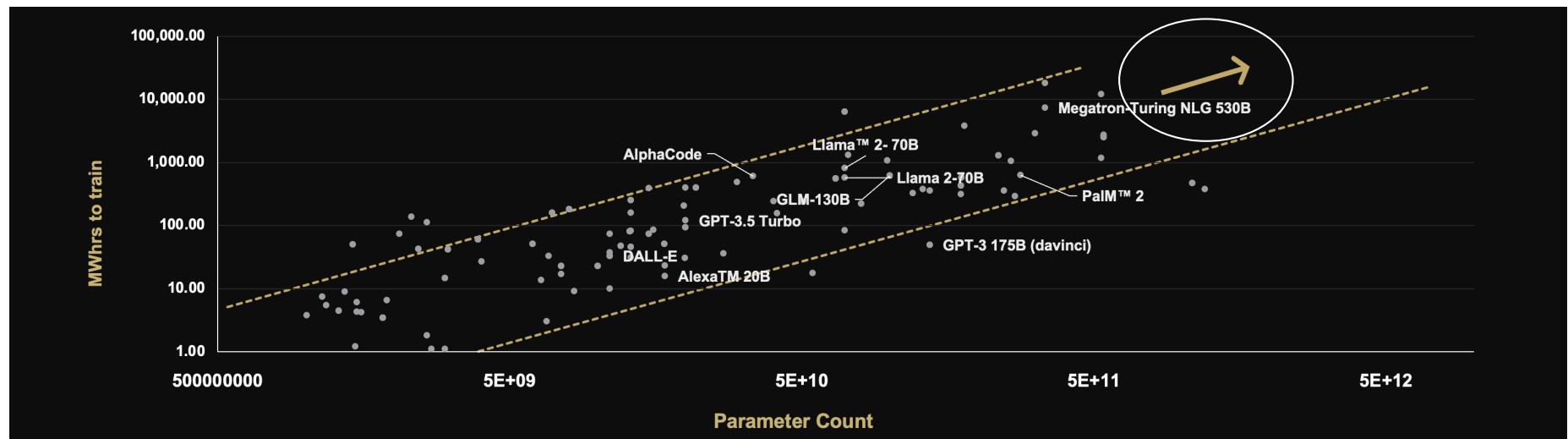
Frontier AI Models



Exponential growth in model sizes driving massive increase in energy required for training

Inference reasoning and agentic AI are driving more energy demand across CPUs and GPUs

Frontier AI Models



The key message

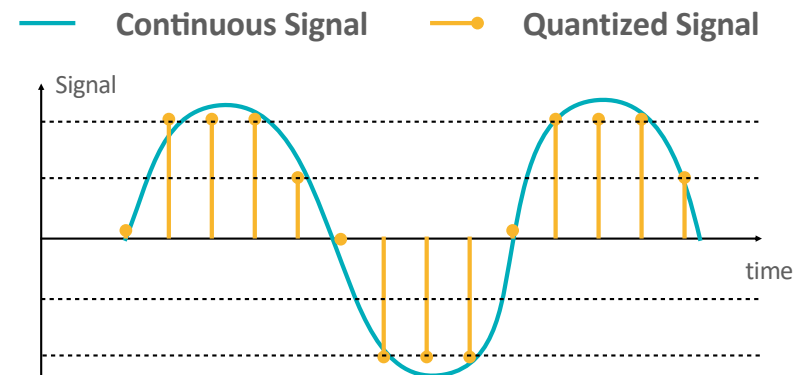
The arrow pointing upper-right emphasizes the trajectory: as parameter counts grow from 10^9 to 10^{12} , training energy grows from ~ 1 MWh to $\sim 10,000$ MWh — a **four-order-of-magnitude** increase for roughly a **three-order-of-magnitude** increase in parameters. This super-linear scaling is precisely why quantization matters: if you can train or run inference in FP8 instead of FP32, you directly cut into this energy budget.

Agenda

1. Numeric ***data types*** used in the modern computing systems, including integers and floating-point numbers.
2. Learn the basic concept of ***neural network quantization***
3. Quantization approaches:
 1. **Post-Training Quantization (PTQ)** that quantizes a floating-point neural network model, including weight quantization, activation quantization, and bias quantization.
 2. **Quantization-Aware Training (QAT)** that emulates inference-time quantization during the training/fine-tuning and recover the accuracy.

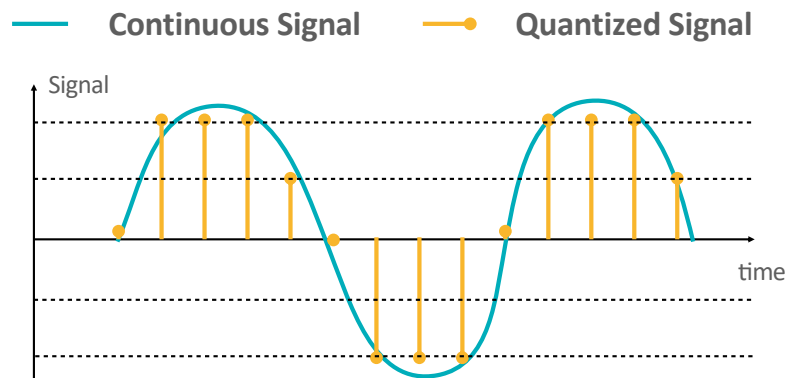
1	1	0	0	1	1	1	1
x	x	x	x	x	x	x	x

$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

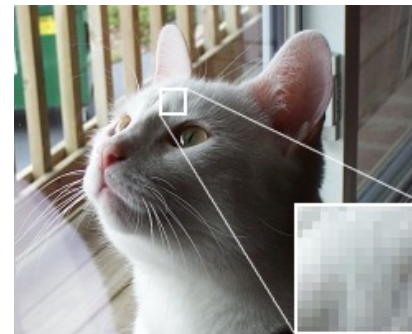


What is Quantization?

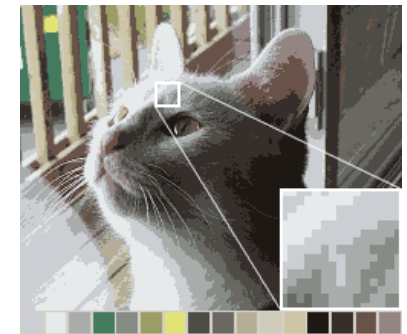
Quantization is the process of constraining an input from a continuous or otherwise large set of values to a discrete set.



Original Image



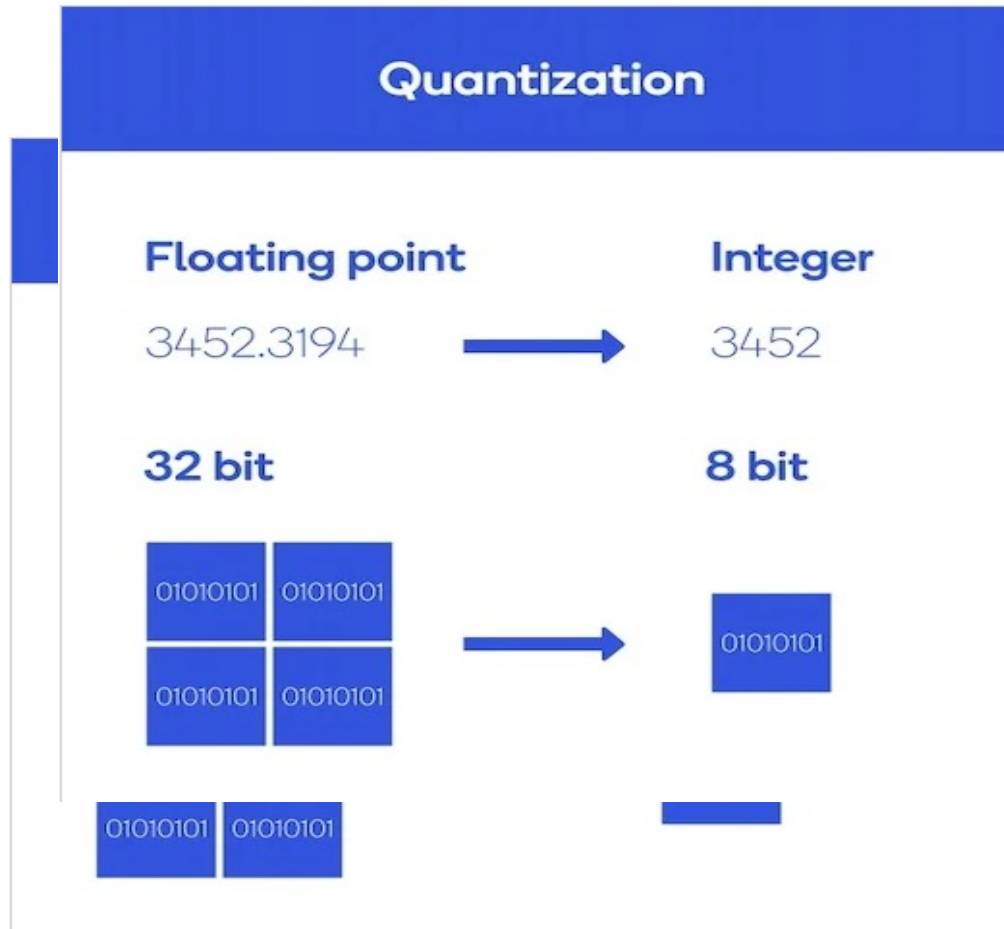
16-Color Image



[Images](#) are in the public domain.

- [Wikipedia: Quantization](#)

What is Quantization?



Quantizing an image - Example



8 bits/pixel



4 bits/pixel



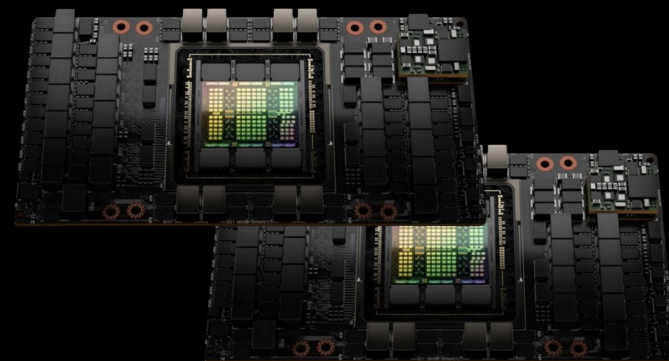
2 bits/pixel



DeepSeek-R1:
671B parameters



Original



Quantized

Numerical Data Types

Integer

- Unsigned Integer

- n -bit Range: $[0, 2^n - 1]$

0	0	1	1	0	0	0	1
x	x	x	x	x	x	x	x

$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

- Signed Integer

- Sign-Magnitude Representation

- n -bit Range: $[-2^{n-1} - 1, 2^{n-1} - 1]$
- Both 000...00 and 100...00 represent 0

Sign Bit

1	0	1	1	0	0	0	1
x	x	x	x	x	x	x	x

$$- 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

- Two's Complement Representation

- n -bit Range: $[-2^{n-1}, 2^{n-1} - 1]$
- 000...00 represents 0
- 100...00 represents -2^{n-1}

1	1	0	0	1	1	1	1
x	x	x	x	x	x	x	x

$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

Fixed-Point Number



Integer . Fraction

"Decimal" Point



$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ -2^3 & +2^2 & +2^1 & +2^0 & -2^{-1} & -2^{-2} & -2^{-3} & +2^{-4} \end{array} = 3.0625$$



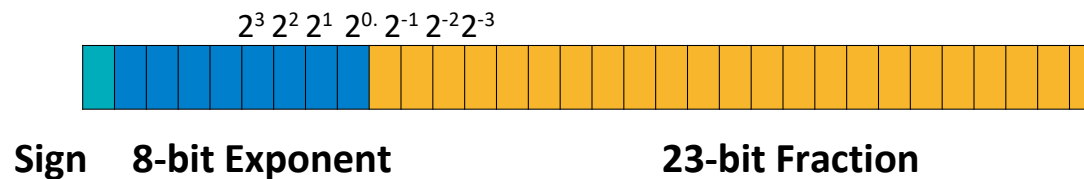
$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ (-2^7 & +2^6 & +2^5 & +2^4 & +2^3 & +2^2 & +2^1 & +2^0) \times 2^{-4} = 49 \times 0.0625 = 3.0625 \end{array}$$

(using 2's complement representation)

Floating-Point Number

- The workhorse of deep learning

Example: 32-bit floating-point number in IEEE 754

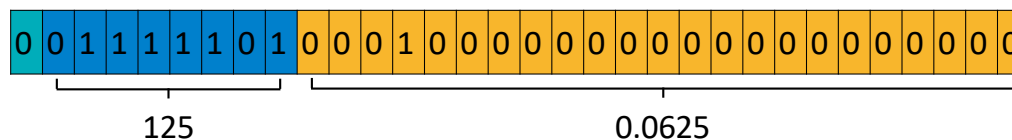


$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

(significant / mantissa)

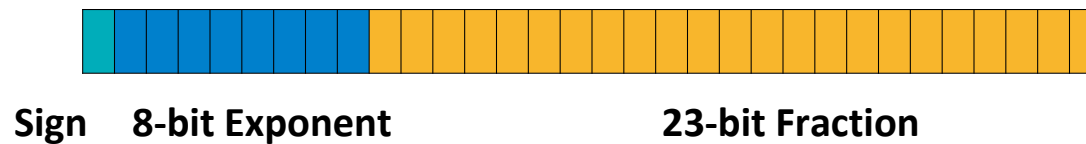
← Exponent Bias = 127 = $2^{8-1}-1$

$$0.265625 = 1.0625 \times 2^{-2} = (1 + \underline{0.0625}) \times 2^{\underline{125}-127}$$

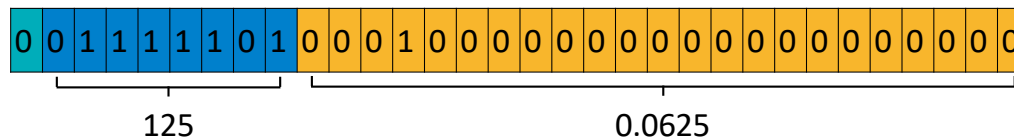


Floating-Point Number

Example: 32-bit floating-point number in IEEE 754

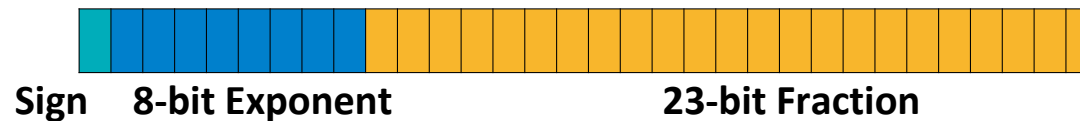


How to represent 0?



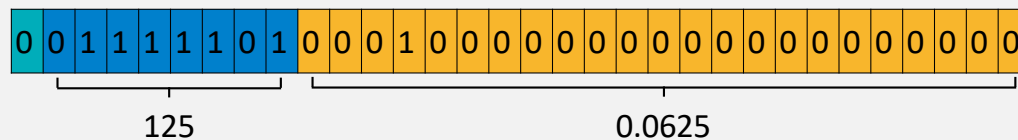
Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Normal Numbers, Exponent $\neq 0$

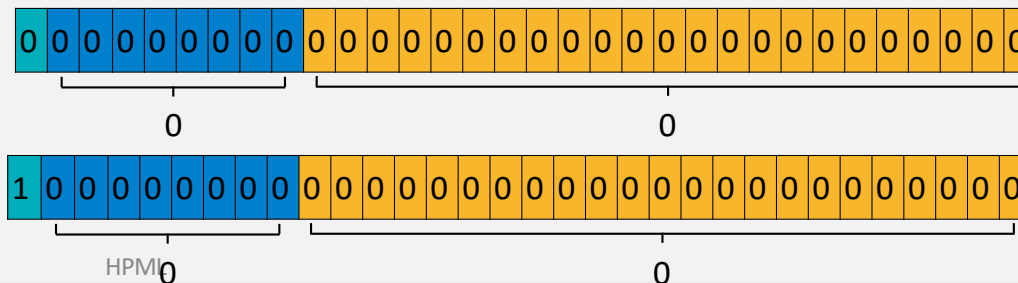
$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$



$$\Rightarrow 0.265625 = 1.0625 \times 2^{-2} = (1 + 0.0625) \times 2^{125-127}$$

Subnormal Numbers, Exponent = 0

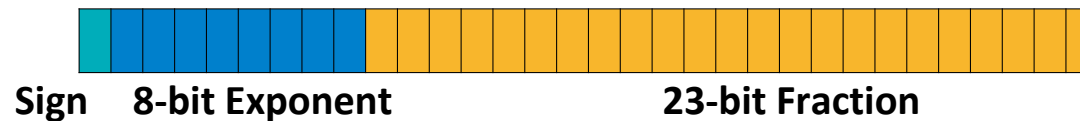
Instead of $(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{0-127}$, we force it to be $(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$



$$\Rightarrow 0 = 0 \times 2^{-126}$$

Floating-Point Number

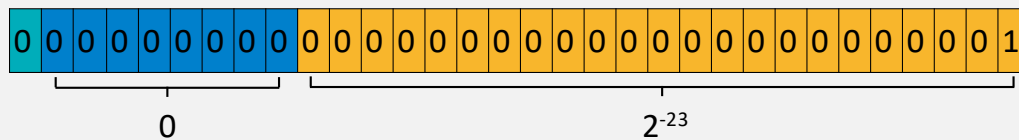
Example: 32-bit floating-point number in IEEE 754



Subnormal Numbers, Exponent = 0

$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$$

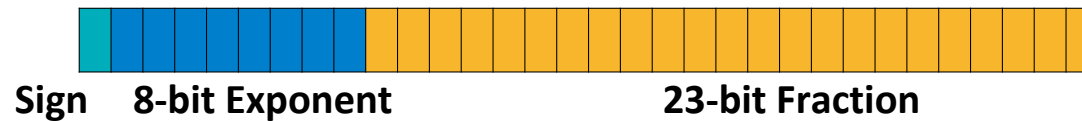
Smallest positive subnormal value that can be represented:



➡ $2^{-149} = 2^{-23} \times 2^{-126}$

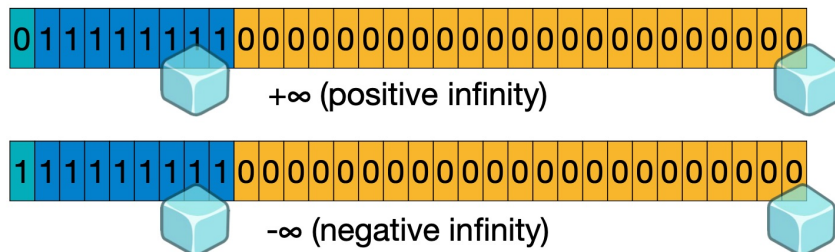
Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



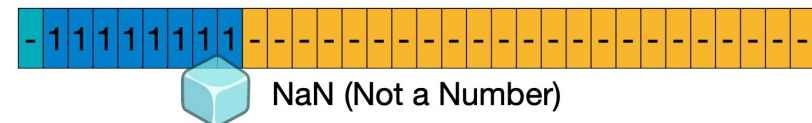
$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

(Normal Numbers, Exponent $\neq$ 0)



$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$$

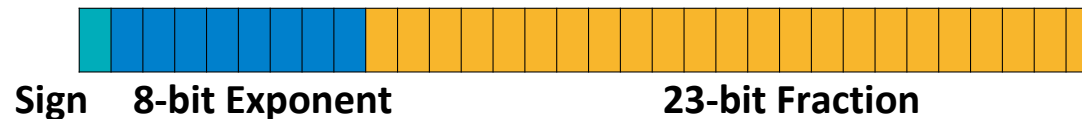
(Subnormal Numbers, Exponent=0)



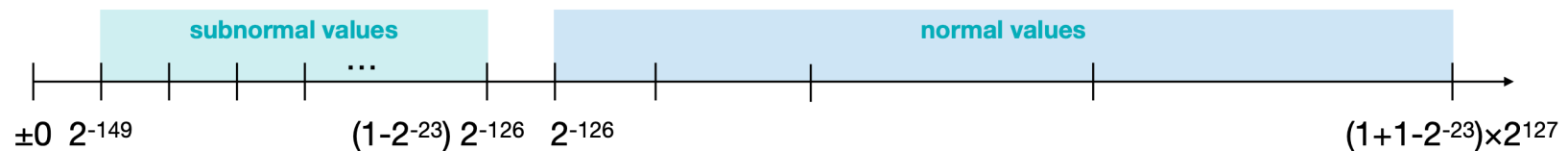
much waste. Revisit in fp8.

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754

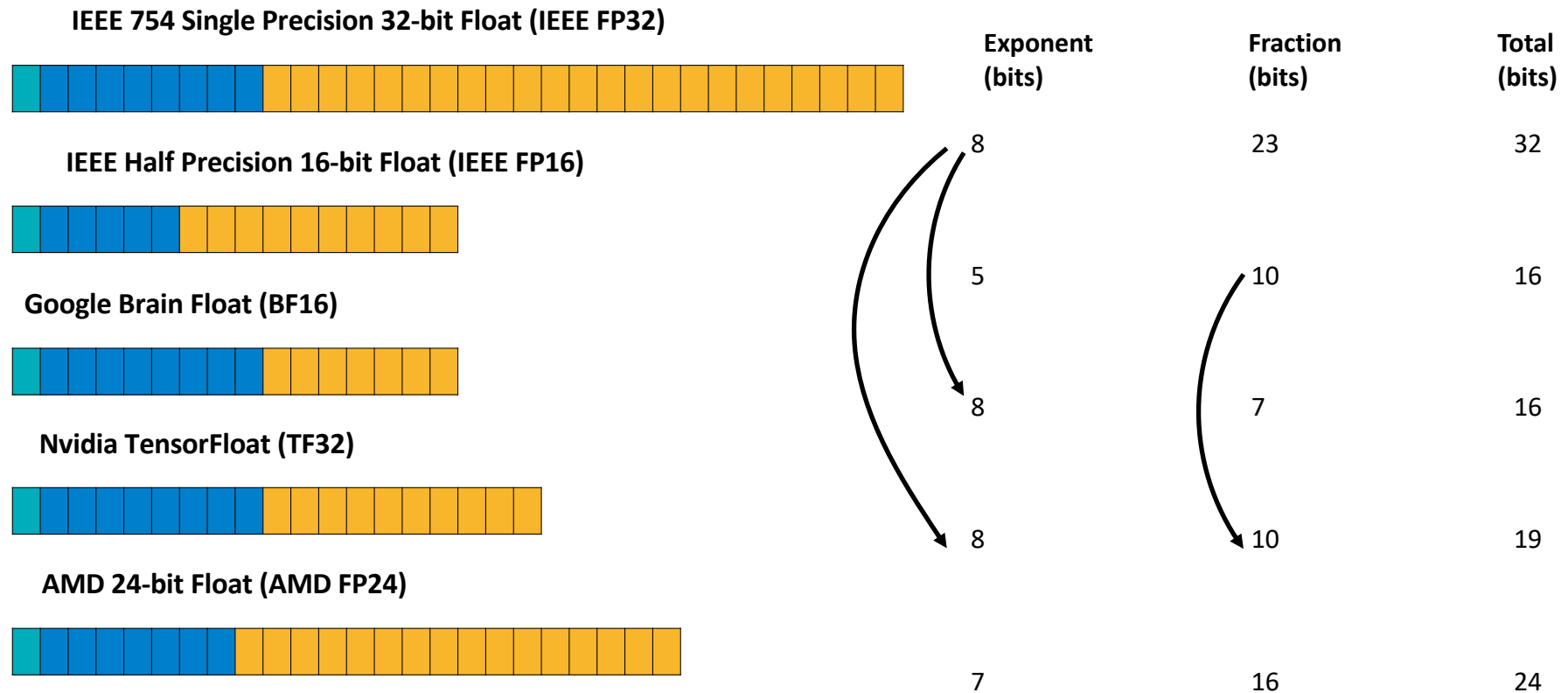


Exponent	Fraction=0	Fraction≠0	Equation
00 _H = 0 	±0	subnormal	$(-1)^{\text{sign}} \times \mathbf{\text{Fraction}} \times 2^{1-127}$
01 _H ... FE _H = 1 ... 254	normal		$(-1)^{\text{sign}} \times (1 + \mathbf{\text{Fraction}}) \times 2^{\mathbf{\text{Exponent}}-127}$
FF _H = 255 	±INF 	NaN	



Floating-Point Number

Exponent Width → Range; Fraction Width → Precision



Floating-Point Number – NVIDIA FP8

Exponent Width → Range; Fraction Width → Precision

- The FP8 datatype supported by H100 is actually 2 distinct datatypes, useful in different parts of the training of neural networks.
- The tradeoff of the increased dynamic range is the lower precision of the stored values.

Nvidia FP8 (E4M3)



- FP8 E4M3 does not have INF, and S.1111.111 is used for NaN.
- The largest FP8 E4M3 normal value is S.1111.110 = 448.
- **Forward activations and weights require more precision**, so the E4M3 datatype is best used during forward pass.

Nvidia FP8 (E5M2)



- FP8 E5M2 have INF (S.11111.002) and NaN (S.11111.XX).
- The largest FP8 E5M2 normal value is S.11110.11 = 57344.
- **In the backward pass, gradients** flowing through the network typically are less susceptible to the loss of precision but **require a higher dynamic range**. Therefore, they are best stored using the E5M2 data format.

Reference: https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html

Floating-Point Number – NVIDIA FP4

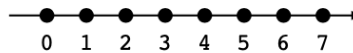
- Introduced in Blackwell

Exponent Width → Range; Fraction Width → Precision

INT4

S			
0	0	0	1
0	1	1	1

-1, -2, -3, -4, -5, -6, -7, -8
0, 1, 2, 3, 4, 5, 6, 7



-1, -2, -3, -4, -5, -6, -7, -8
0, 1, 2, 3, 4, 5, 6, 7

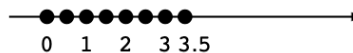
=1

=7

FP4 (E1M2)

S	E	M	M
0	0	0	1
0	1	1	1

-0, -0.5, -1, -1.5, -2, -2.5, -3, -3.5
0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5



-0, -1, -2, -3, -4, -5, -6, -7 $\times 0.5$
0, 1, 2, 3, 4, 5, 6, 7 $\times 0.5$

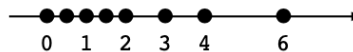
$=0.25 \times 2^{1-0} = 0.5$

$=(1+0.75) \times 2^{1-0} = 3.5$

FP4 (E2M1)

S	E	E	M
0	0	0	1
0	1	1	1

-0, -0.5, -1, -1.5, -2, -3, -4, -6
0, 0.5, 1, 1.5, 2, 3, 4, 6



-0, -1, -2, -3, -4, -6, -8, -12 $\times 0.5$
0, 1, 2, 3, 4, 6, 8, 12 $\times 0.5$

$=0.5 \times 2^{1-1} = 0.5$

$=(1+0.5) \times 2^{3-1} = 1$

no inf, no NaN

FP4 (E3M0)

S	E	E	E
0	0	0	1
0	1	1	1

-0, -0.25, -0.5, -1, -2, -4, -8, -16
0, 0.25, 0.5, 1, 2, 4, 8, 16



-0, -1, -2, -4, -8, -16, -32, -64 $\times 0.25$
0, 1, 2, 4, 8, 16, 32, 64 $\times 0.25$

$=(1+0) \times 2^{1-3} = 0.25$

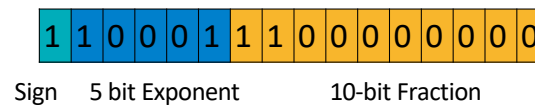
$=(1+0) \times 2^{7-3} = 16$

no inf, no NaN

<https://efficientml.ai>

Numeric Data Types

- **Question:** What is the following IEEE half precision (IEEE FP16) number in decimal?



Exponent Bias = 15_{10}

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-15}$$



Exponent Bias = $15 = 2^{5-1}-1$

- Sign: -
- Exponent: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
- Exponent- Bias: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
- Fraction: $1100000000_2 = 0.75_{10}$
- Decimal Answer = $-(1 + 0.75) \times 2^2 = -1.75 \times 2^2 = -7.0_{10}$

Numeric Data Types

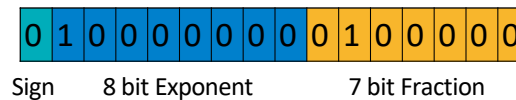
- **Question:** What is the decimal 2.5 in Brain Float (BF16)?

$$2.5_{10} = 1.\underline{25}_{10} \times 2^1$$

Exponent Bias = 15_{10}

- Sign: +
- Exponent Binary: $1_{10} + 15_{10} = 16_{10} = 00010000_2$
- Fraction Binary: $0.25_{10} = 0100000_2$

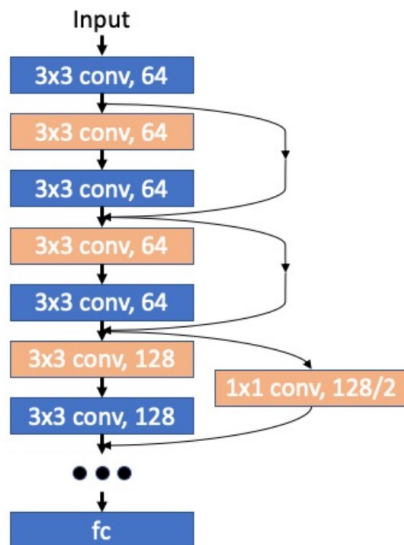
- Binary Answer



Neural Network Quantization

For any given trained neural network:

- Store weights in low bits (INT8)
- Compute calculations in low bits



Quantization Analogy

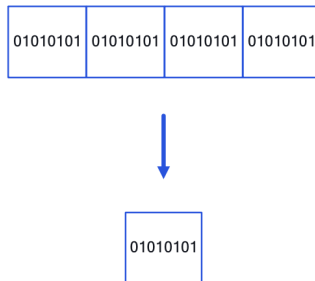
Use fewer bits to represent each pixel in an image



Significant Benefits with Quantization

Memory usage

8-bit versus 32-bit weights and activations stored in memory



Power consumption

Significant reduction in energy for both computations and memory access

Add energy (pJ)		Mem access energy (pJ)	
INT8	FP32	Cache (64-bit)	
0.03	0.9	8KB	10
30X energy reduction		32KB	20
		1MB	100
Mult energy (pJ)		DRAM	
INT8	FP32	1300-	2600
0.2	3.7	Up to 4X energy reduction	
18.5X energy reduction			

Latency

With less memory access and simpler computations, latency can be reduced



Silicon area

Integer math or less bits require less silicon area compared to floating point math and more bits

Add area (μm ²)	
INT8	FP32
36	4184
116X area reduction	
Mult area (μm ²)	
INT8	FP32
282	7700
27X area reduction	

Why Quantization Matters

The Challenge

Modern LLMs have billions of parameters

LLaMA 4 Maverick: 400B+ parameters
(Meta, 2025)

LLaMA 3.1: 405B parameters

FP32 model with 7B params = ~28GB
memory

Key Benefits

2-4x memory reduction

2-5x faster inference

Enables edge device deployment

Reduces operational costs

Market: \$150B+ projected by 2030
(McKinsey, 2025)

Design Space: What/How/When Do We Quantize?

What

- Weights only
- Weights + activations
- (Sometimes) gradients & optimizer states

How

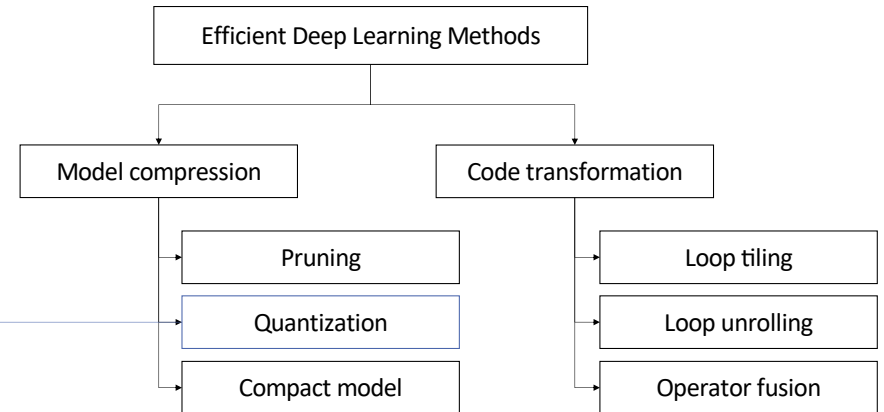
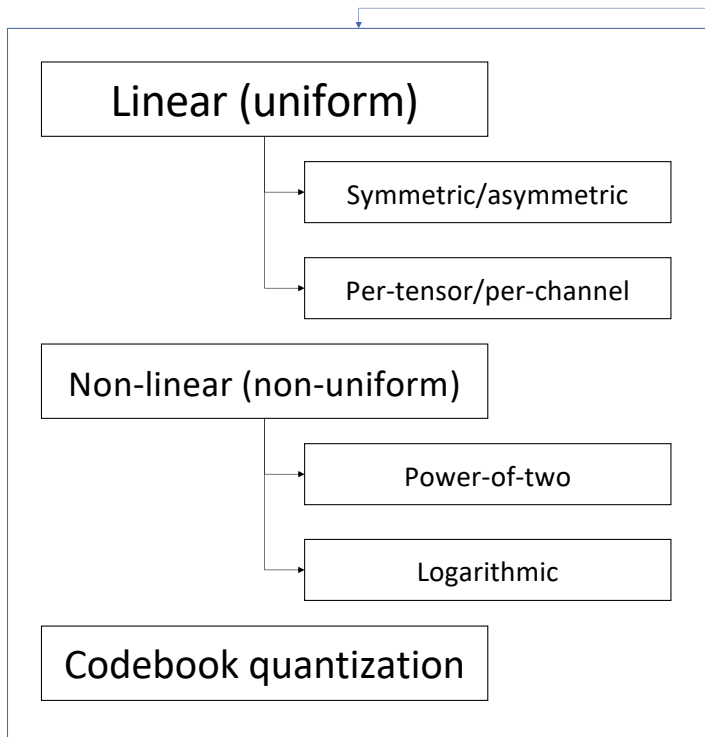
- Symmetric vs asymmetric
- Per-tensor vs per-channel vs per-group scaling

When:

- **PTQ** (Post-Training Quantization): no/low retraining
- **QAT** (Quantization-Aware Training): simulate quantization during training

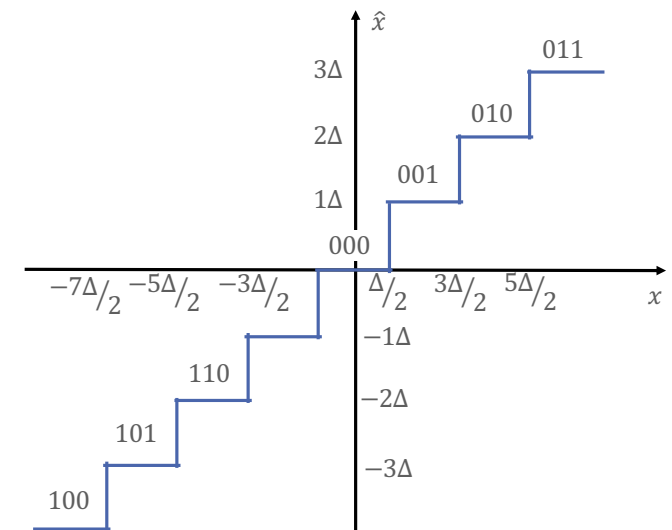
How to Quantize?

How to quantize?



Linear (uniform) quantization

- Linear implies constant step size Δ
- Step size is determined by the range
 - Symmetric or asymmetric mode
 - Per-tensor or per-channel
- $\hat{x}$ the quantized version of x :
$$\hat{x} = \text{round}(x/\Delta) \cdot \Delta$$

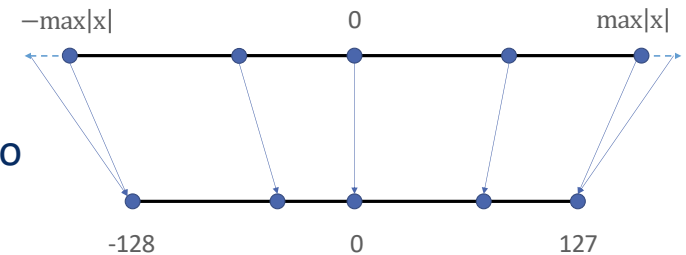


3 bits quantization example.

The 3-bit example shows values mapped to 8 levels (-4 to 3 in two's complement).

Symmetric mode

- Quantization range is symmetric around 0
- Step size given by:
 - $\Delta = \frac{2 \cdot \max|x|}{2^n - 1}$
- Conventional fixed-point format when step size is a power-of-two
- KEY POINTS:
 - Symmetric quantization for weights is standard in most frameworks.
 - The downside: if your distribution is very asymmetric (e.g., all positive values after ReLU), symmetric quantization wastes half the integer range on values that don't appear.
 - This motivates asymmetric quantization for activations.



Asymmetric mode

Useful when the distribution of values is **not** symmetric around 0 (e.g. after ReLU)

Step size given by:

$$\Delta = \frac{\max x - \min x}{2^n - 1}$$

Use a zero-point z to shift the distribution:

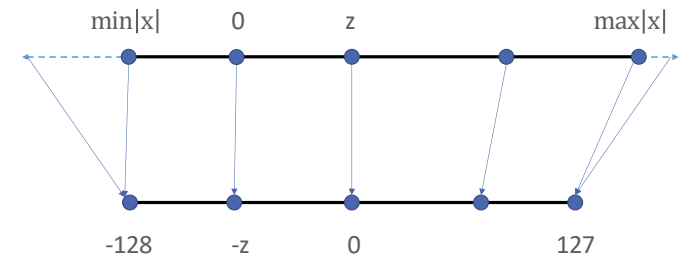
$$\hat{x} = \text{round}\left(\frac{x}{\Delta} - z\right) \cdot \Delta + z$$

In practice the zero-point is chosen such that zero is exactly representable by an integer in the quantized domain

Asymmetric quantization is often used in applications requiring higher precision, such as NLP tasks, or for layers with high sensitivity, like embeddings.

More details on asymmetric quantization matrix multiplication:

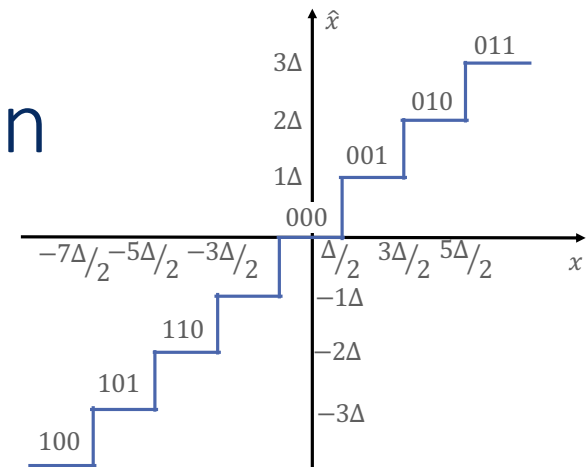
<https://github.com/google/gemmlowp/blob/master/doc/quantization.md#implementation-of-quantized-matrix-multiplication>



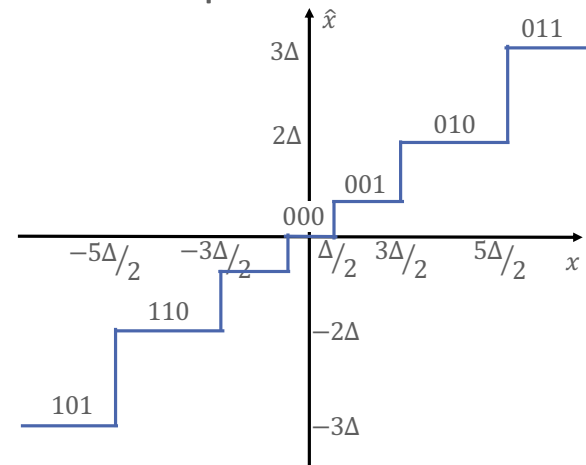
Non-linear (non-uniform) quantization

- Non-linear implies variable step size Δ
- Variability follows a regular pattern:
 - Power-of-two step
 - Logarithmic
- Concept:

Assume weights/activations are non-uniformly distributed. More sampling points where many values reside increases the entropy of the quantized signal.



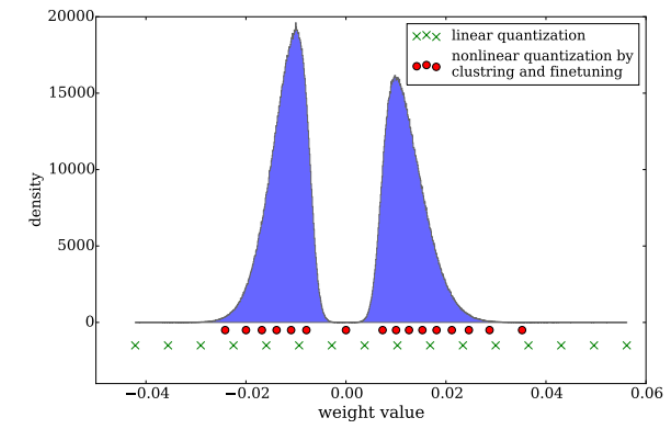
Linear quantization



Non-linear quantization

Codebook quantization

- Useful when distribution of values is not normally distributed
- Quantized values (codewords) are represented by a codebook
- Codewords are selected using a clustering algorithm or training process
- Codewords may be scalars or vectors
- Note: requires hardware support for lookup



Codebook example

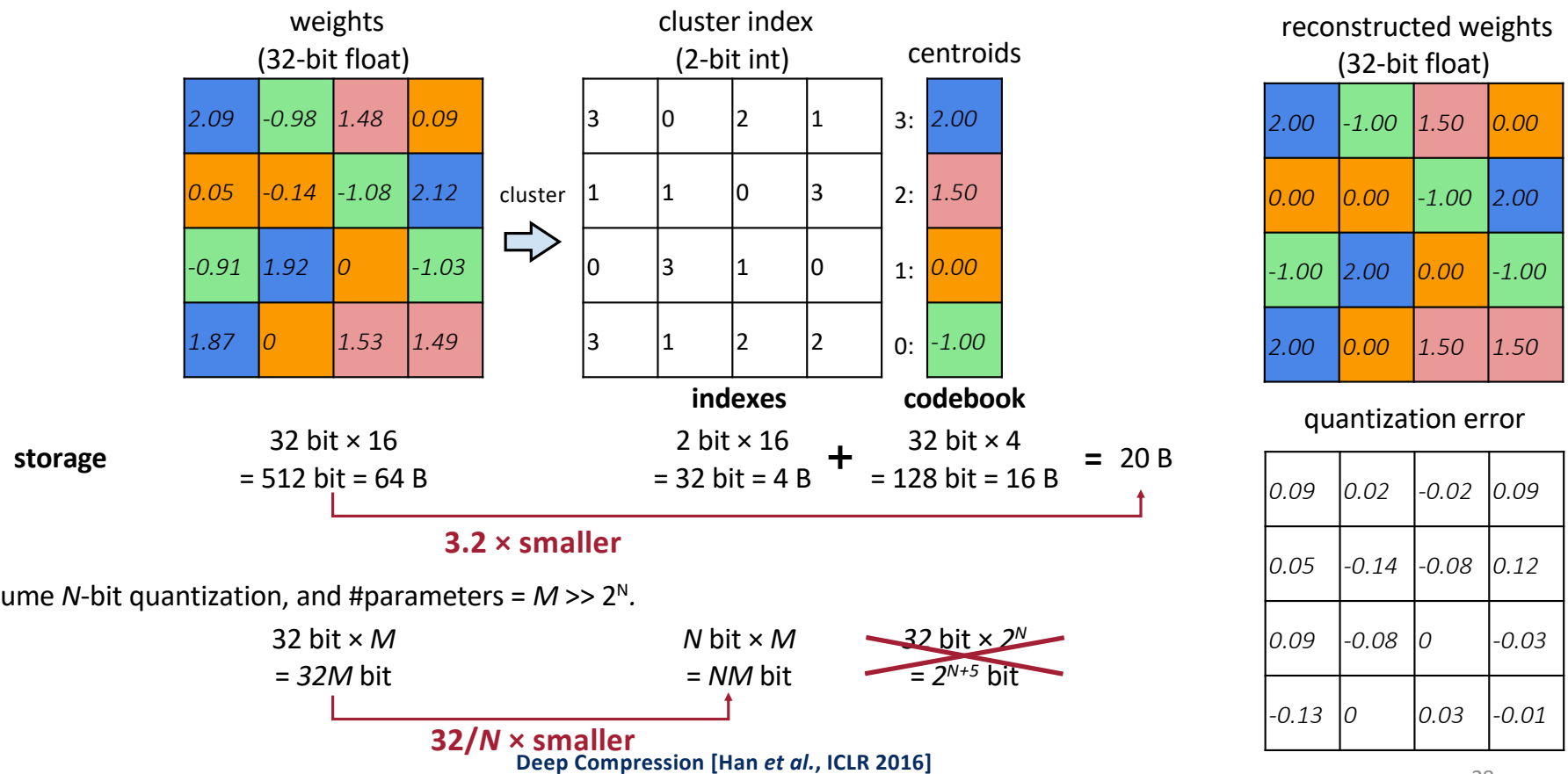
Index	Value	Interval
0	-0.07	[-0.08, -0.04]
1	0	[-0.04, 0.05]
2	0.01	[0.05, 0.011]
3	0.012	[0.011, 0,013]

Example: K-Means-based Weight Quantization – Example of Codebook Quantization

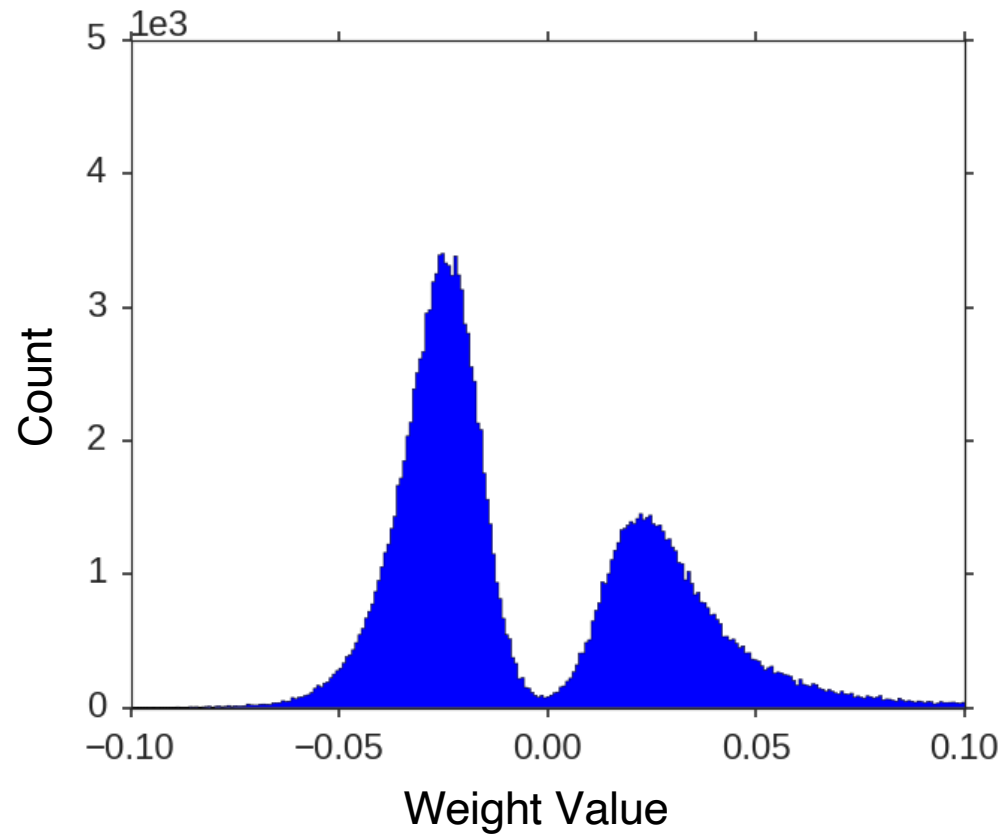
weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Example: K-Means-based Weight Quantization (Inference Pass)

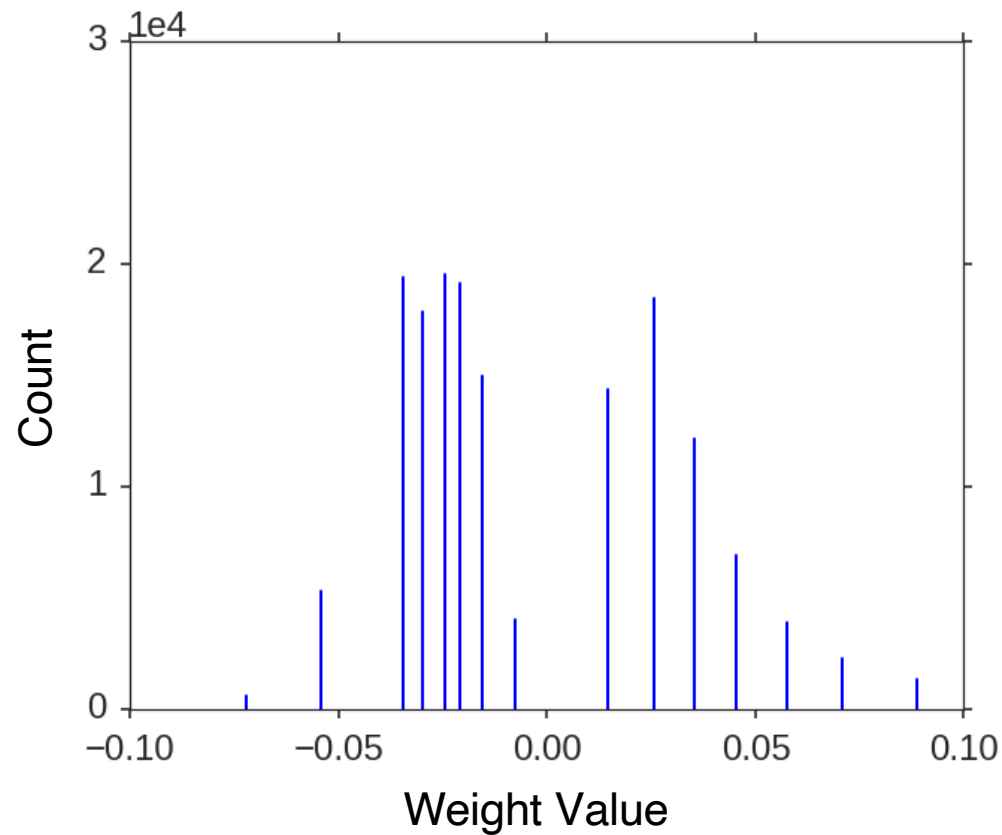


Before Quantization: Continuous Weight



- Deep Compression [Han *et al.*, ICLR 2016]
<https://efficientml.ai>

After Quantization: Discrete Weight



- Deep Compression [Han *et al.*, ICLR 2016]
<https://efficientml.ai>

How to quantize? - Summary

Scheme	Step size	Step size pattern
Linear	Fixed	Regular
Non-linear	Variable	Regular
Codebook	Variable	Irregular

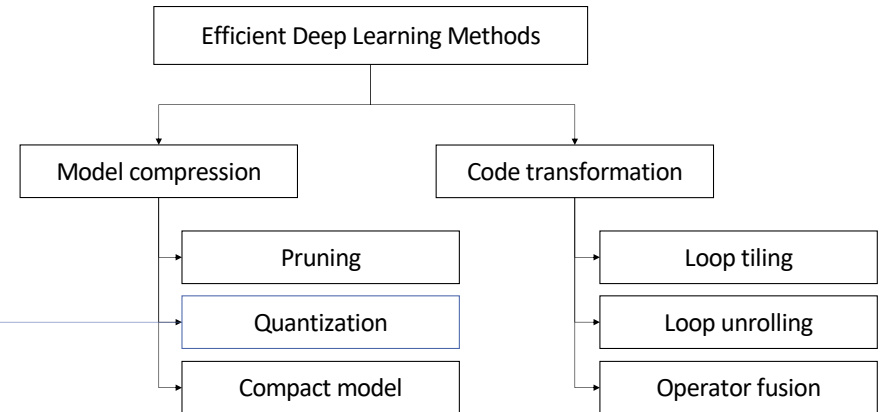
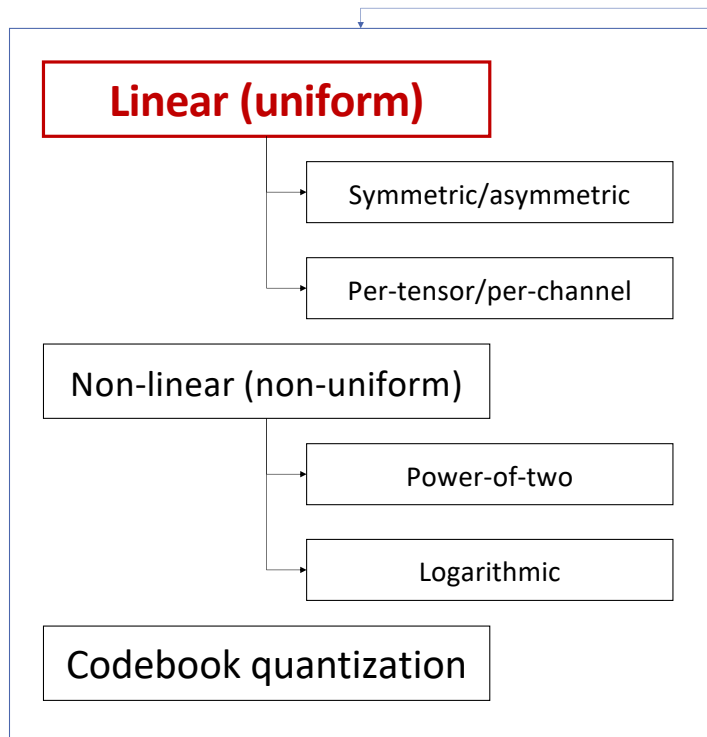
Summary – Linear Quantization Methods

Quantization Type	Description	Pros	Cons	Industry Adoption
Linear Uniform Symmetric	Uses a single scale factor for both positive and negative values around zero, with uniform steps.	- Simpler and faster computations. - Supported by most hardware.	- Not ideal for skewed data distributions, potentially leading to higher quantization error.	High Common in computer vision and edge applications where hardware supports symmetric int8 quantization.
Linear Uniform Asymmetric	Allows separate scale factors or offsets for positive and negative ranges, adapting to non-centered data.	- Handles skewed distributions better than symmetric quantization. - Suitable for NLP and sensitive layers.	- Adds complexity in hardware and software. - Increased computational overhead due to varying zero points.	Moderate to High Widely used in NLP and models with data skew where precision is crucial.

Summary – Non-Linear and Cookbook Quantization Methods

Quantization Type	Description	Pros	Cons	Industry Adoption
Non-Linear (Non-Uniform)	Quantization steps are not evenly spaced, often using logarithmic or other non-linear distributions.	- More efficient for highly skewed or non-linear data distributions. - Can reduce quantization error.	- Complex to implement and not widely supported on hardware. - Potential accuracy trade-offs.	Moderate Used in specialized cases like audio, signal processing, or hardware with custom support.
Cookbook Quantization	Custom, layer-specific quantization settings tuned for each layer or specific hardware.	- High level of accuracy with fine-grained control. - Tailored for specific hardware capabilities.	- High implementation complexity. - Limited standardization and scalability.	Low Primarily used in research or experimental setups where precise control is necessary.

How to quantize?



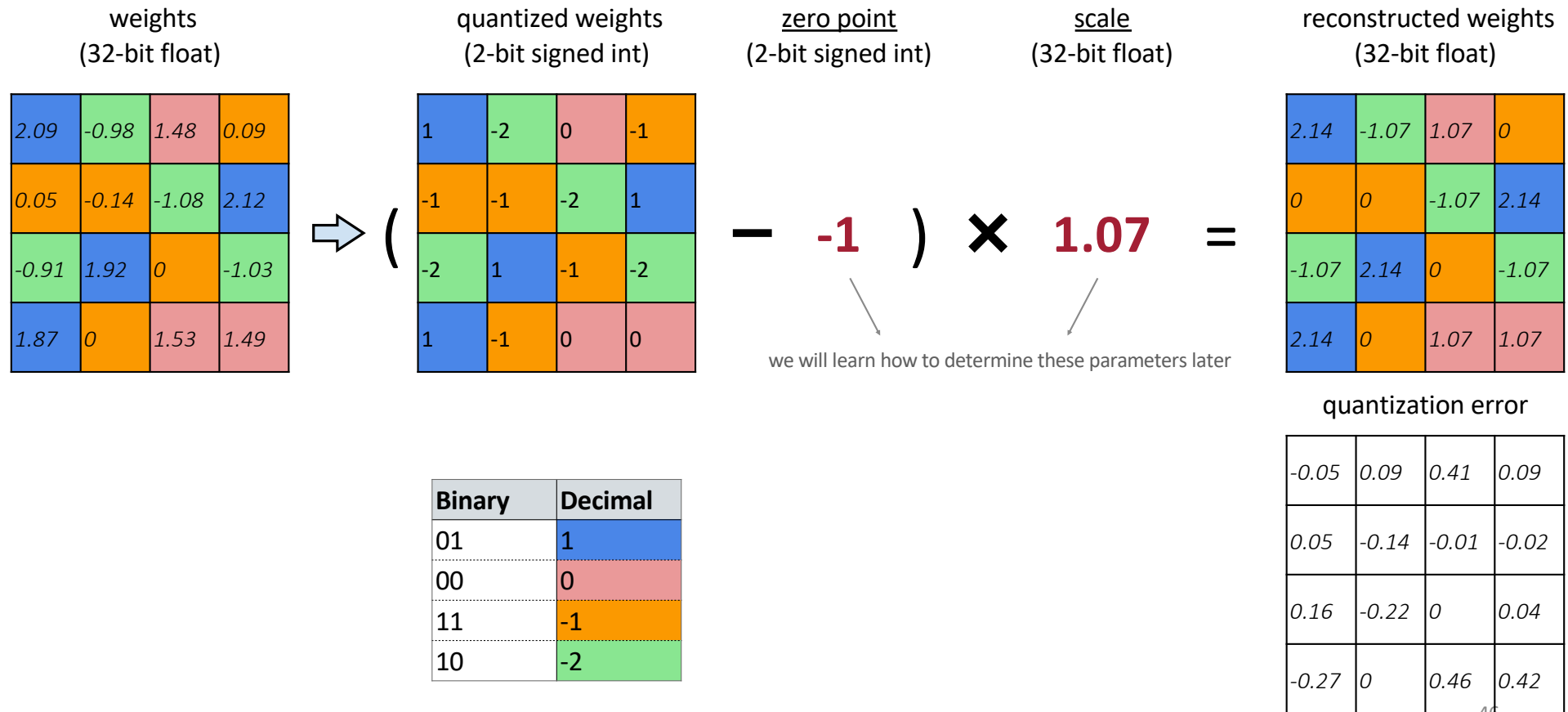
What is Linear Quantization?

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

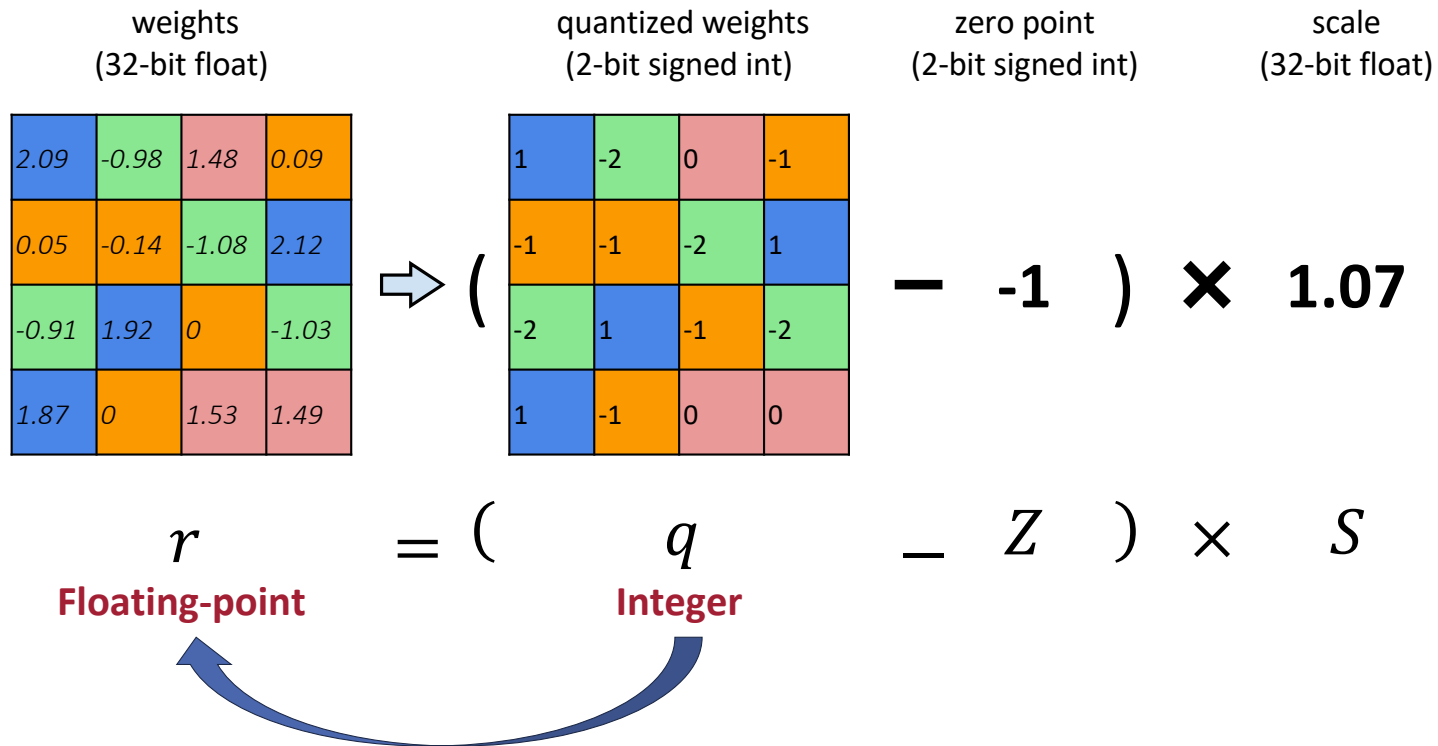
What is Linear Quantization?

An affine mapping of integers to real numbers



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

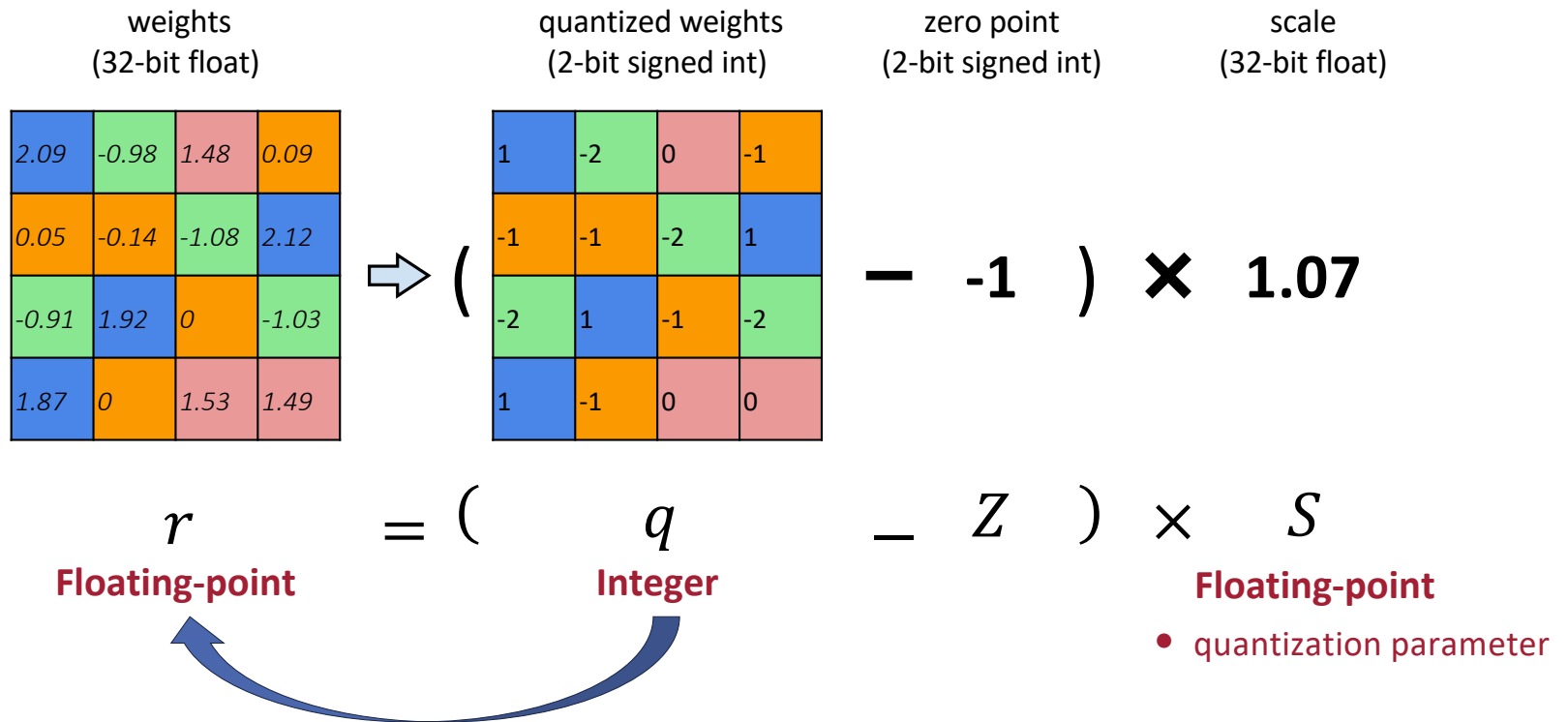


Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

<https://efficientml.ai>

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

<https://efficientml.ai>

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

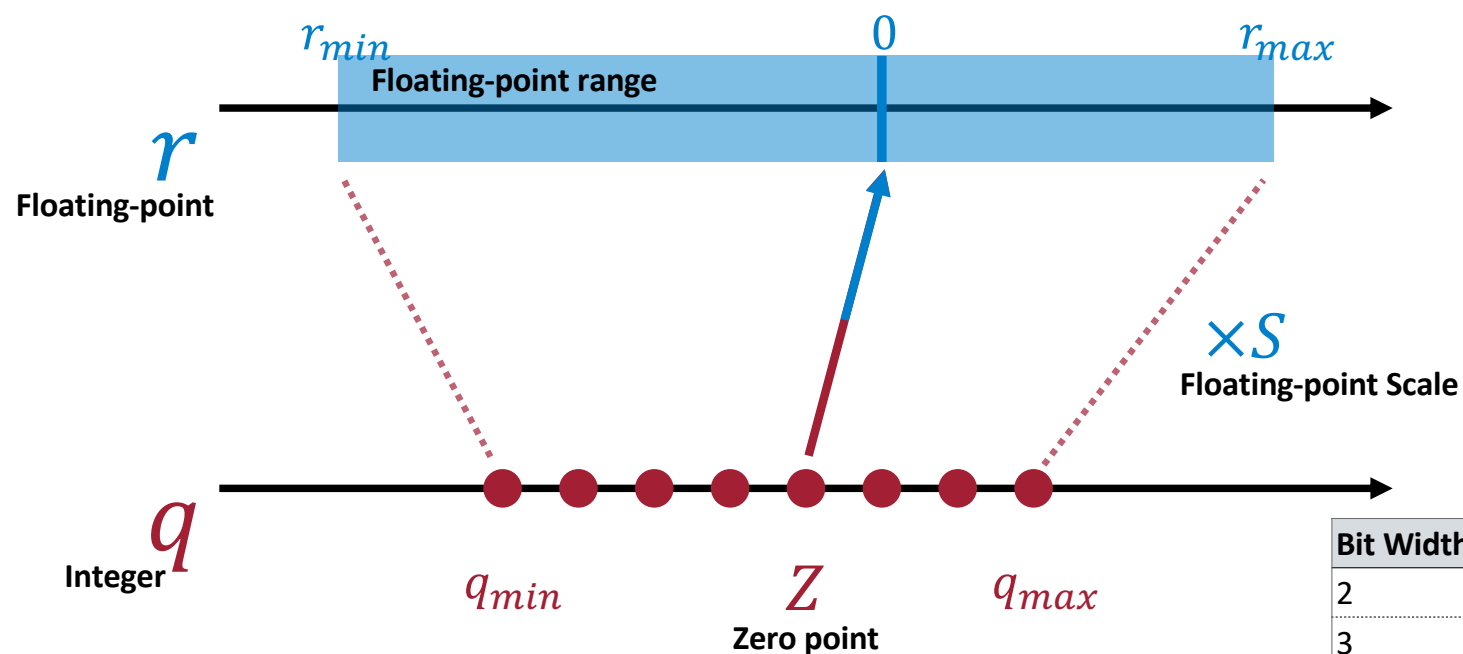
weights (32-bit float)	quantized weights (2-bit signed int)	zero point (2-bit signed int)	scale (32-bit float)																																
<table border="1"> <tr><td>2.09</td><td>-0.98</td><td>1.48</td><td>0.09</td></tr> <tr><td>0.05</td><td>-0.14</td><td>-1.08</td><td>2.12</td></tr> <tr><td>-0.91</td><td>1.92</td><td>0</td><td>-1.03</td></tr> <tr><td>1.87</td><td>0</td><td>1.53</td><td>1.49</td></tr> </table>	2.09	-0.98	1.48	0.09	0.05	-0.14	-1.08	2.12	-0.91	1.92	0	-1.03	1.87	0	1.53	1.49	<table border="1"> <tr><td>1</td><td>-2</td><td>0</td><td>-1</td></tr> <tr><td>-1</td><td>-1</td><td>-2</td><td>1</td></tr> <tr><td>-2</td><td>1</td><td>-1</td><td>-2</td></tr> <tr><td>1</td><td>-1</td><td>0</td><td>0</td></tr> </table>	1	-2	0	-1	-1	-1	-2	1	-2	1	-1	-2	1	-1	0	0	-1	1.07
2.09	-0.98	1.48	0.09																																
0.05	-0.14	-1.08	2.12																																
-0.91	1.92	0	-1.03																																
1.87	0	1.53	1.49																																
1	-2	0	-1																																
-1	-1	-2	1																																
-2	1	-1	-2																																
1	-1	0	0																																

$$\begin{array}{ccccccc}
 r & = & (& q & - & Z &) \times S \\
 \text{Floating-point} & & & \text{Integer} & & \text{Integer} & \text{Floating-point}
 \end{array}$$

- quantization parameter
- allow real number $r=0$ be exactly representable by a quantized integer Z
- quantization parameter

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

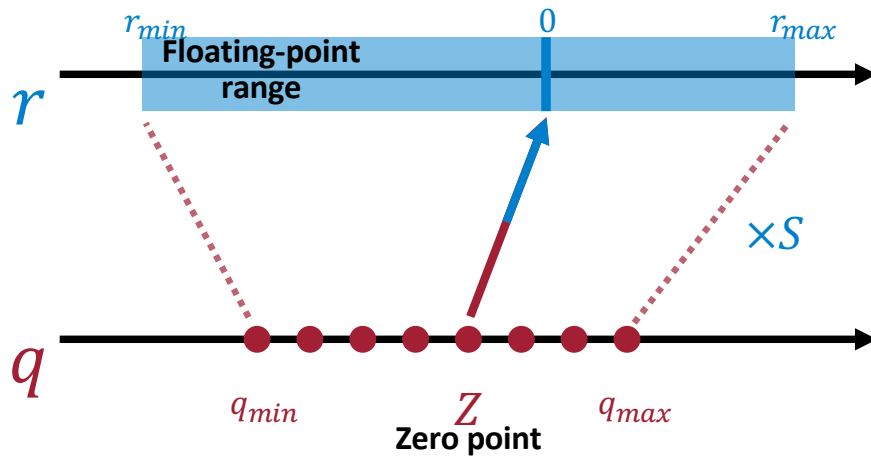


Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

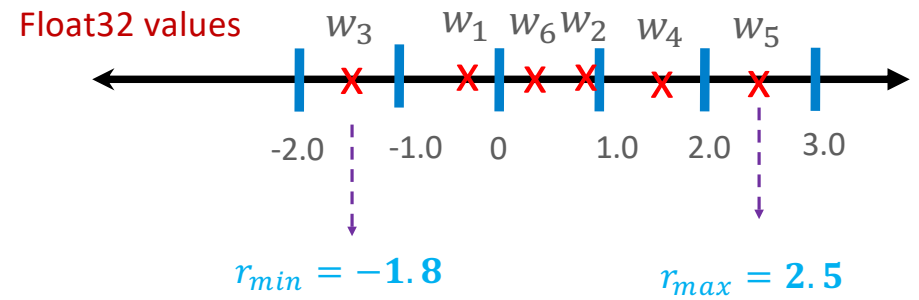
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

<https://efficientml.ai>

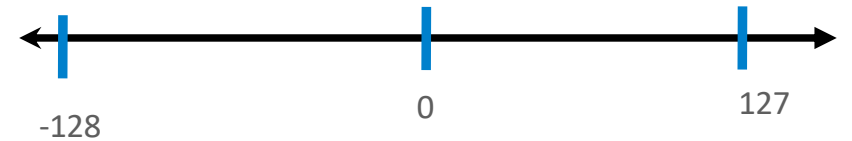
Illustration



$$W = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \\ w_5 & w_6 \end{bmatrix}$$



Int8 values, bit width, b=8

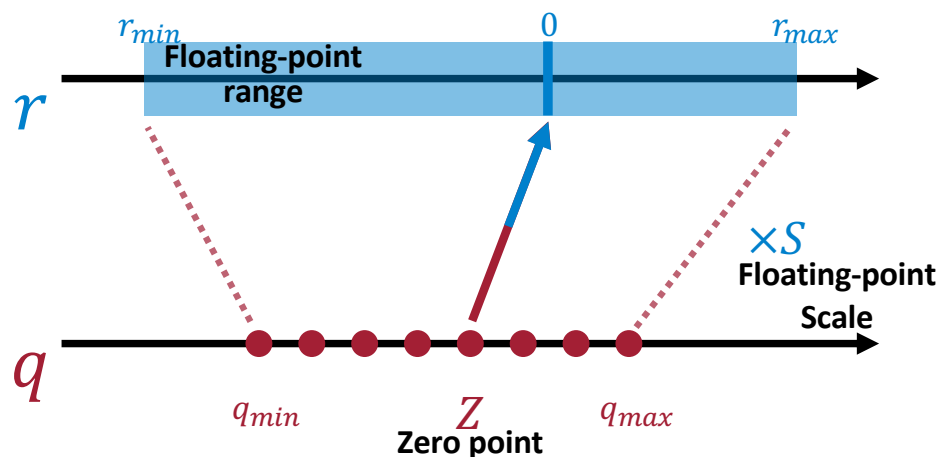


The goal is to map the real values in the range $[r_{min}, r_{max}]$ to the integers in $[-2^{b-1}, 2^{b-1} - 1]$

$[-128, 127]$ When b=8 for Int8

How to calculate the Scale of Linear Quantization?

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



The scale factor is commonly represented as a floating-point number and specifies the *step size* of the quantizer.

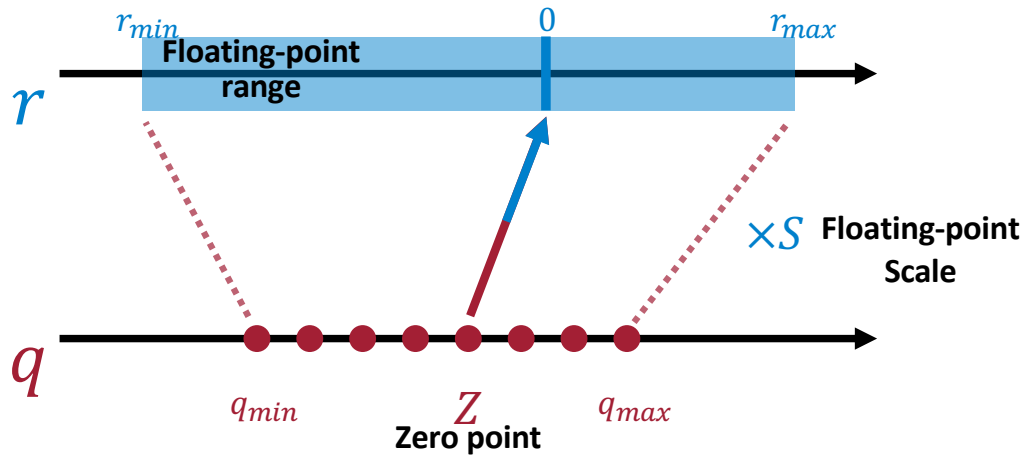
$$\begin{aligned} r_{max} &= S(q_{max} - Z) \\ r_{min} &= S(q_{min} - Z) \end{aligned}$$

$$\downarrow$$
$$r_{max} - r_{min} = S(q_{max} - q_{min})$$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

How to calculate the Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



The zero point is an integer that ensures that the real zero is quantized without error. This is important to ensure that common operations like zero padding or ReLU do not induce quantization error.

$$r_{min} = S(q_{min} - Z)$$

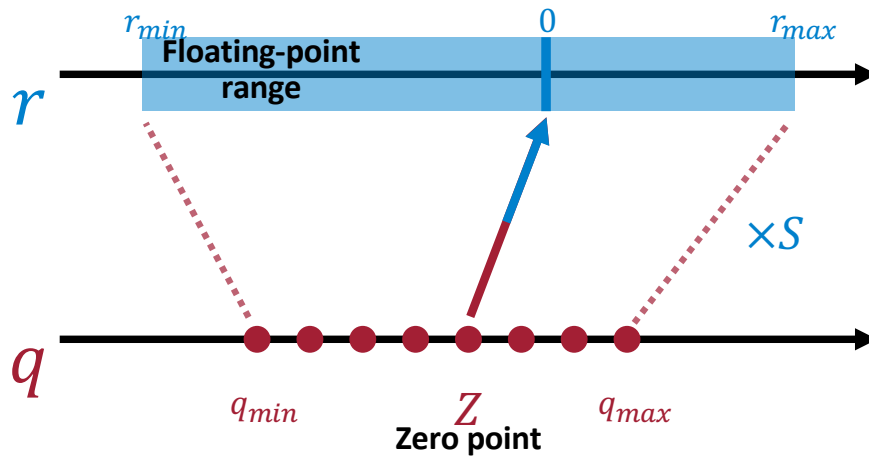
↓

$$Z = q_{min} - \frac{r_{min}}{S}$$

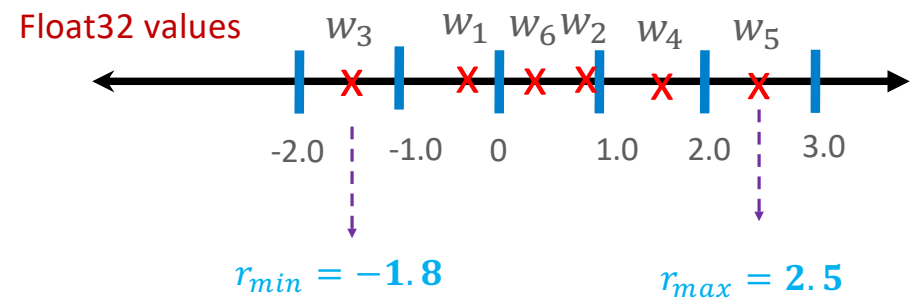
↓

$$Z = \text{round}\left(q_{min} - \frac{r_{min}}{S}\right)$$

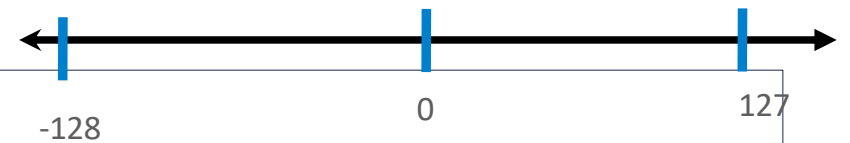
Illustration



$$W = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \\ w_5 & w_6 \end{bmatrix}$$



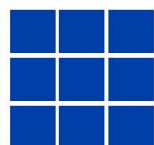
Int8 values, bit width, b=8



Example:

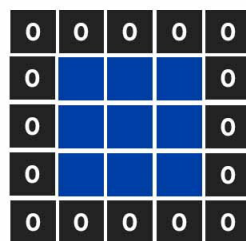
- For INT8: $q_{min} = -128, q_{max} = 127$.
- Calculate $S = (2.5 - (-1.8)) / (127 - (-128)) = 4.3 / 255 \approx 0.01686$.
- Calculate $Z = \text{round}(-(-1.8)/0.01686) + (-128) = \text{round}(106.8) - 128 = 107 - 128 = -21$.
- So, *float 0.0 maps to integer -21*.
- This is exactly what PyTorch computes during INT8 quantization.

The Importance of expressing Zero Correctly



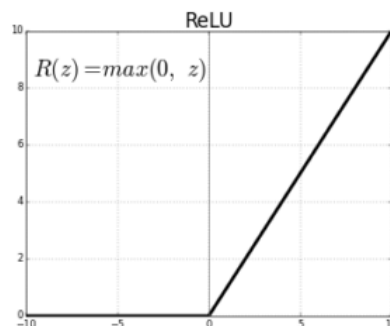
Input Image

Applying padding
of 1 on 3X3

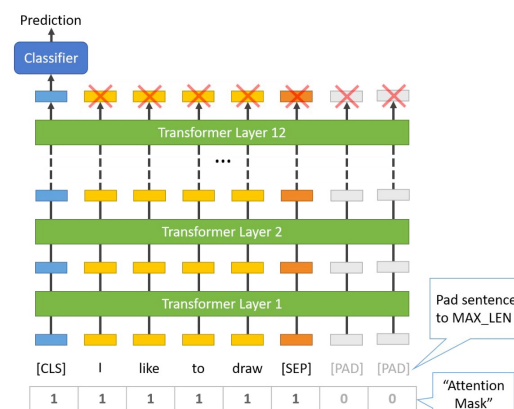


Padded Image

In CNNs, zero-padding, where zeros are added to the input matrix, is essential to preserve the spatial dimensions after convolution



ReLU activation function outputs zero for any negative input and is widely used for its simplicity and effectiveness in introducing non-linearity.



Attention mask in transformers

Attention masks often use zeros to indicate positions that should not be attended to (e.g., padding in sentences)

Summary

Quantization

$$q_{max} = 2^{b-1} - 1 \quad q_{min} = -2^{b-1}$$

X_q is the quantized value of X

$$X_q = \text{Clamp} \left(\text{round} \left(\frac{X}{S} \right) + Z, q_{min}, q_{max} \right)$$

where,

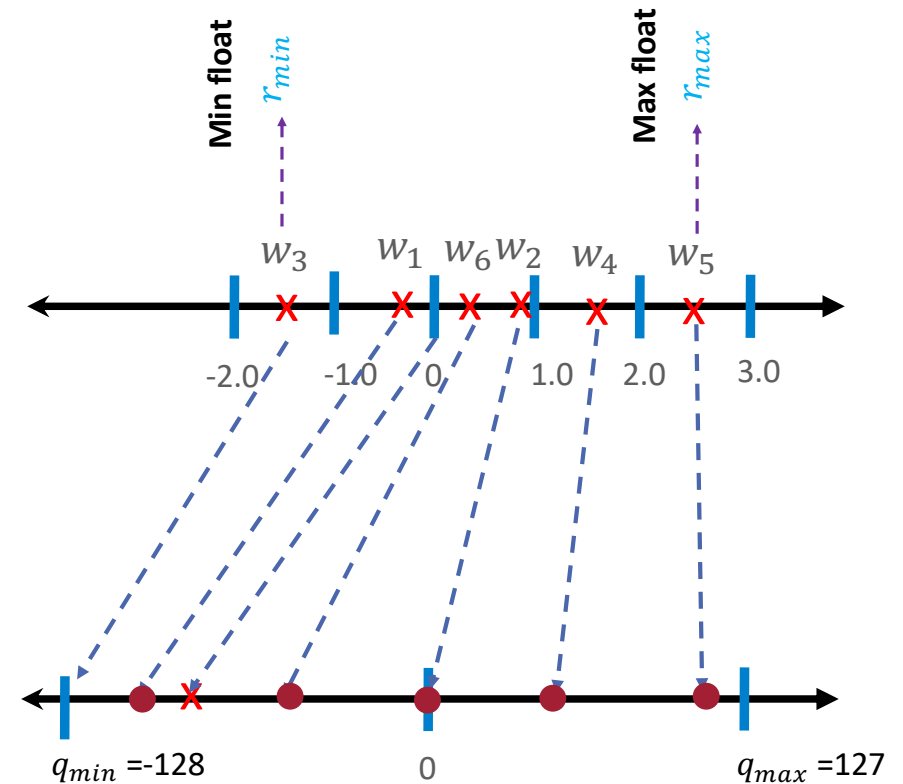
where

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} \quad \dots \quad \text{clamp}(a, b, x) = \begin{cases} a & x \leq a \\ x & a \leq x \leq b \\ b & x \geq b \end{cases}$$

Dequantization

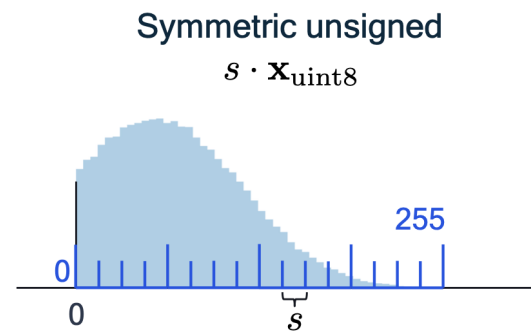
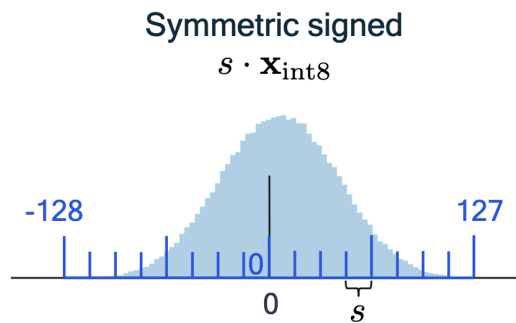
$$X \approx S \times (X_q - Z)$$

Where, Z is the Zero point and S is the scale

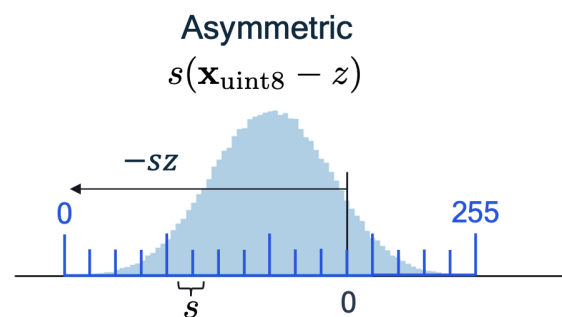


Different Types of quantization

- Symmetric, asymmetric, signed and unsigned quantization

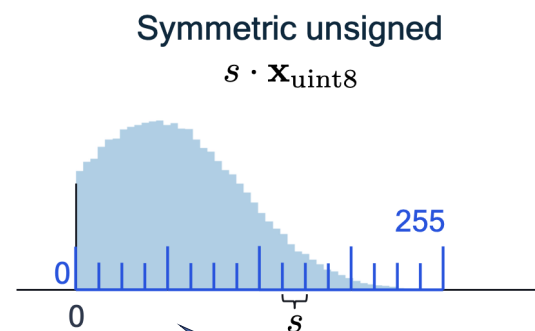
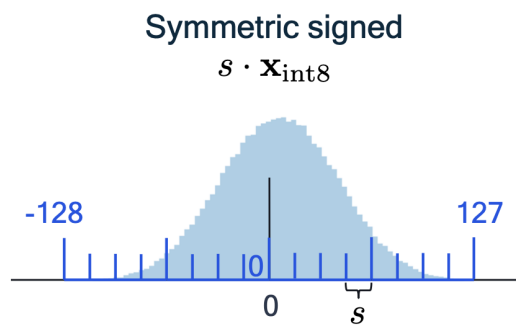


Integer Grid in blue
Floating point Grid in black
S: scale factor
Z: zero point

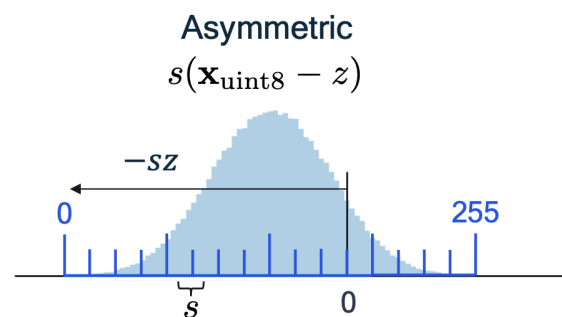


Different Types of quantization

- Symmetric, asymmetric, signed and unsigned quantization



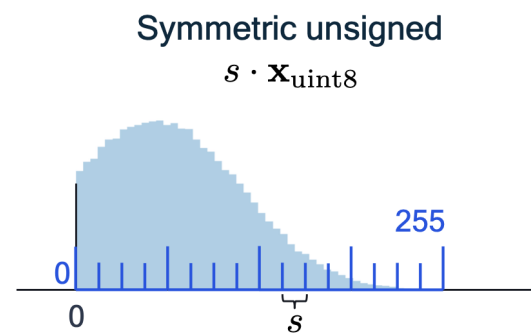
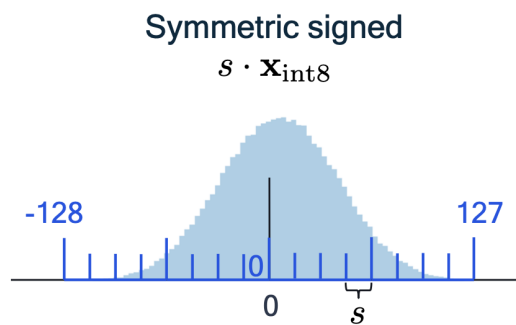
Integer Grid in blue
Floating point Grid in black
S: scale factor
Z: zero point



Symmetric Unsigned
quantization suited for one-
tailed distributions, such as
ReLU activations

Different Types of quantization

- Symmetric, asymmetric, signed and unsigned quantization



Integer Grid in blue
Floating point Grid in black
S: scale factor
Z: zero point

Asymmetric

Symmetric signed quantization
can be chosen for distributions
that are roughly symmetric
about zero

Symmetric Uniform Quantization

- Symmetric quantization is a simplified version of the general asymmetric case

$$\hat{\mathbf{x}} = s \mathbf{x}_{\text{int}}$$

$$\mathbf{x}_{\text{int}} = \text{clamp} \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rfloor; 0, 2^b - 1 \right)$$

for unsigned integers

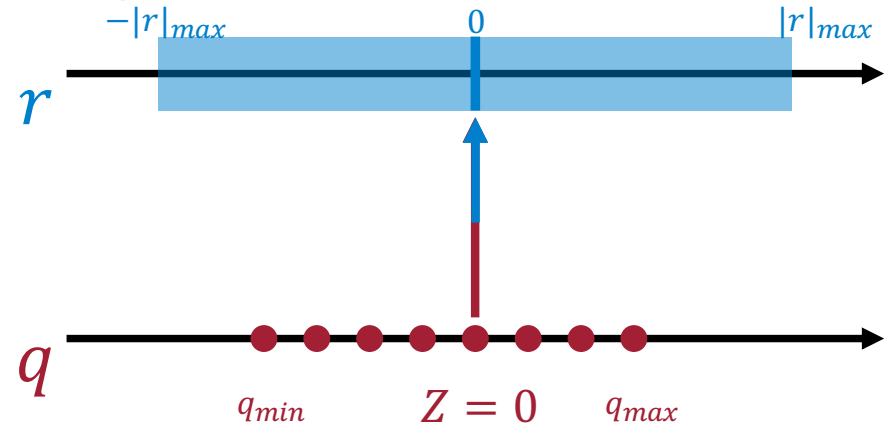
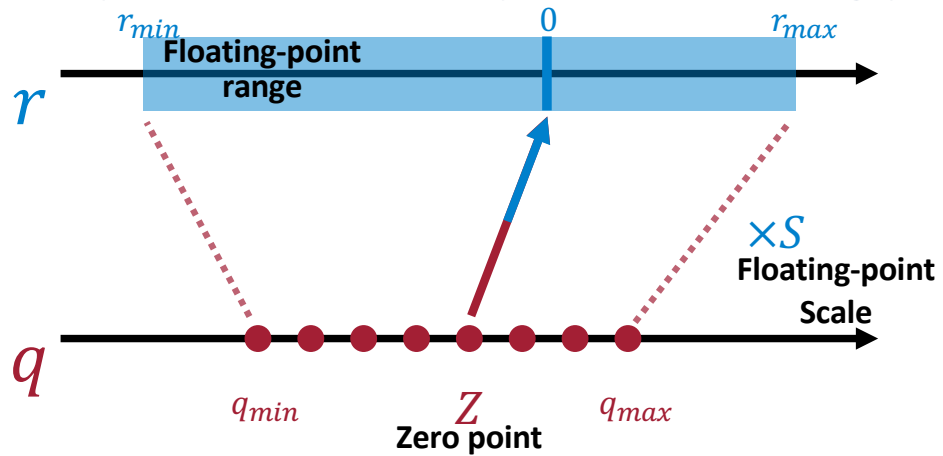
$$\mathbf{x}_{\text{int}} = \text{clamp} \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rfloor; -2^{b-1}, 2^{b-1} - 1 \right)$$

for signed integers

<https://arxiv.org/pdf/2106.08295.pdf>

Symmetric Linear Quantization

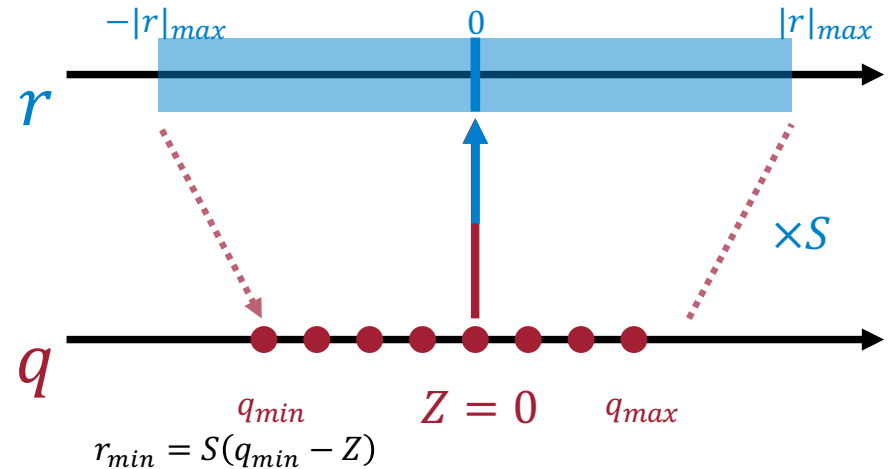
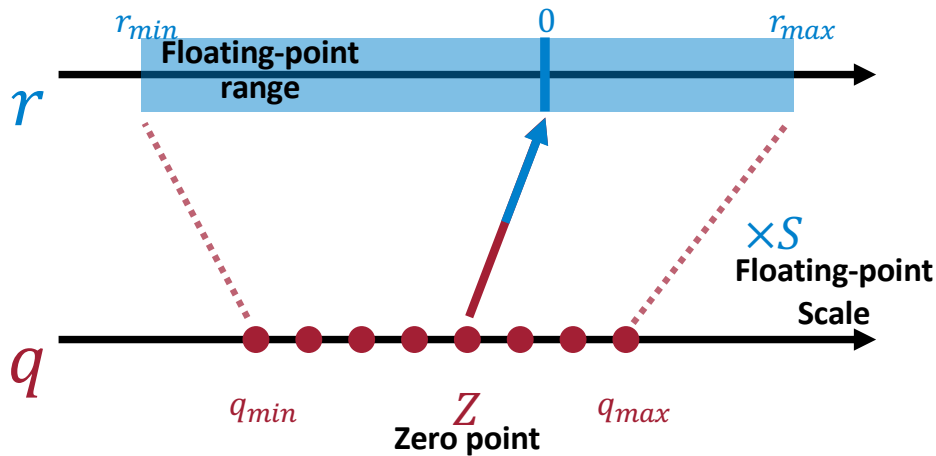
Zero point $Z = 0$ and Symmetric floating-point range



Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Symmetric Linear Quantization

Full range mode



Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$S = \frac{r_{min}}{q_{min} - Z} = \frac{-|r|_{max}}{q_{min}} = \frac{|r|_{max}}{2^{N-1}}$$

- use full range of quantized integers
- example: PyTorch's native quantization

What types of quantization to use?

W : weight matrix

X : input of a layer

Symmetric quantization

$$\begin{aligned} WX &\approx s_W(W_{\text{int}}) s_X(X_{\text{int}}) \\ &= s_W s_X(W_{\text{int}} X_{\text{int}}) \end{aligned}$$

Asymmetric quantization

$$\begin{aligned} WX &\approx s_W(W_{\text{int}} - z_W) s_X(X_{\text{int}} - z_X) \\ &= s_W s_X(W_{\text{int}} X_{\text{int}}) + s_W s_X z_X W_{\text{int}} + s_W z_W s_X z_X + s_W s_X z_W X_{\text{int}} \end{aligned}$$

What types of quantization to use?

W : weight matrix

X : input of a layer

Symmetric quantization

Asymmetric quantization

$$WX \approx s_W(W_{\text{int}}) s_X(X_{\text{int}})$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}})$$

$$WX \approx s_W(W_{\text{int}} - z_W) s_X(X_{\text{int}} - z_X)$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}}) + s_W s_X z_X W_{\text{int}} + s_W z_W s_X z_X + s_W s_X z_W X_{\text{int}}$$

Same calculation

What types of quantization to use?

W : weight matrix

X : input of a layer

Symmetric quantization

Asymmetric quantization

$$WX \approx s_W(W_{\text{int}}) s_X(X_{\text{int}})$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}})$$

Same calculation

$$WX \approx s_W(W_{\text{int}} - z_W) s_X(X_{\text{int}} - z_X)$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}}) + s_W s_X z_X W_{\text{int}} + s_W z_W s_X z_X + s_W s_X z_W X_{\text{int}}$$

Precompute, add to
layer bias

What types of quantization to use?

W : weight matrix

X : input of a layer

Symmetric quantization

Asymmetric quantization

$$WX \approx s_W(W_{\text{int}}) s_X(X_{\text{int}})$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}})$$

Same calculation

$$WX \approx s_W(W_{\text{int}} - z_W) s_X(X_{\text{int}} - z_X)$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}}) + s_W s_X z_X W_{\text{int}} + s_W z_W s_X z_X + s_W s_X z_W X_{\text{int}}$$

Precompute, add to
layer bias

Data-dependent
overhead

Asymmetric weight quantization is equivalent to adding an input channel

What types of quantization to use?

W : weight matrix

X : input of a layer

Symmetric quantization

Asymmetric quantization

$$WX \approx s_W(W_{\text{int}}) s_X(X_{\text{int}})$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}})$$

Same calculation

$$WX \approx s_W(W_{\text{int}} - z_W) s_X(X_{\text{int}} - z_X)$$

$$= s_W s_X(W_{\text{int}} X_{\text{int}}) + s_W s_X z_X W_{\text{int}} + s_W z_W s_X z_X + s_W s_X z_W X_{\text{int}}$$

Precompute, add to
layer bias

Data-dependent
overhead

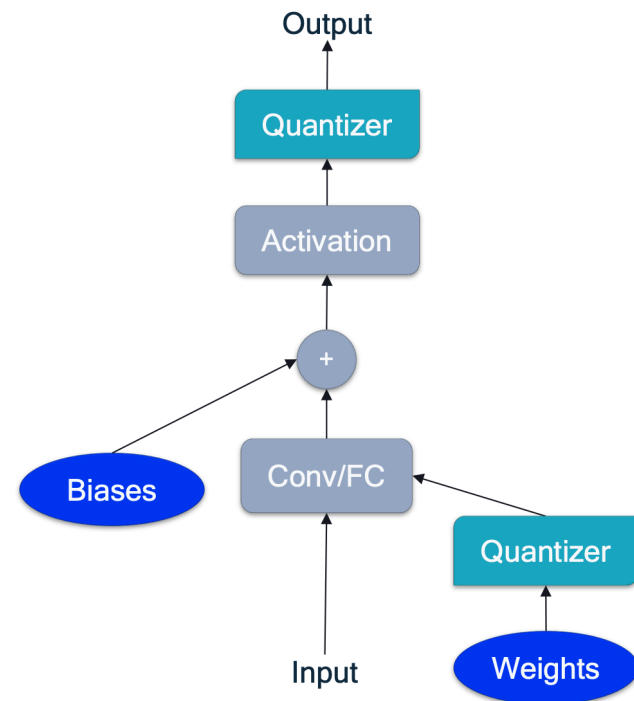
Asymmetric weight quantization is equivalent to adding an input channel

Symmetric weights (W) and asymmetric activations (X) are more hardware efficient

Simulating Quantization

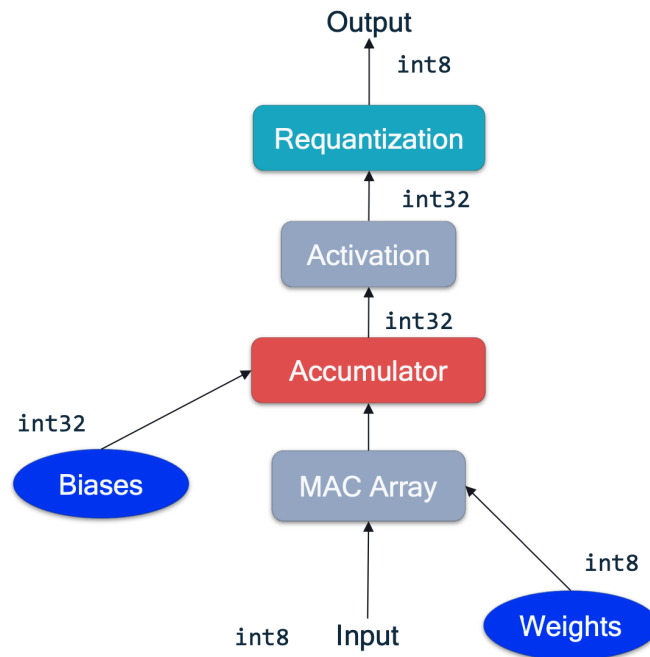
Why simulate quantization?

- We simulate fixed-point operations with floating-point numbers using general purpose hardware (e.g., CPU, GPU)
- This simulation is achieved by introducing simulated quantization operations (quantizers) to the compute graph.
- Quantization simulation benefits:
 - Enables GPUs acceleration
 - No need for dedicated kernel
 - Test various quantization option and bit-widths



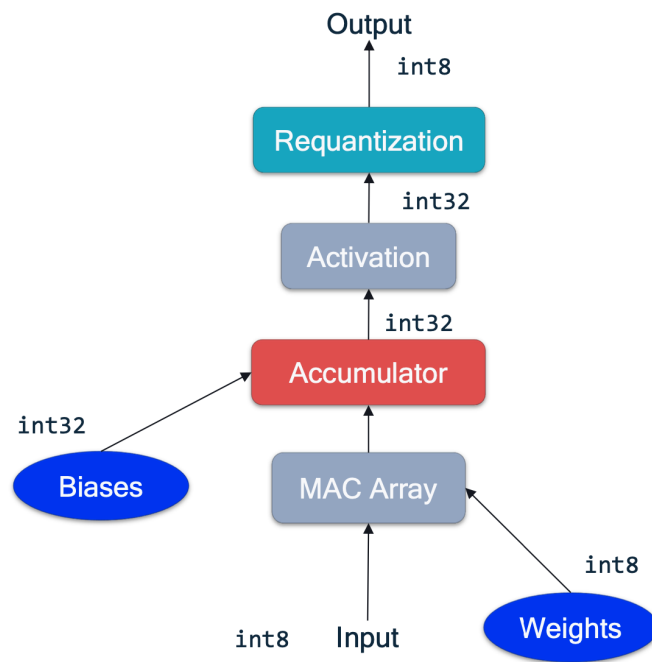
On-device vs. Simulated Quantization

On-device fixed-point inference

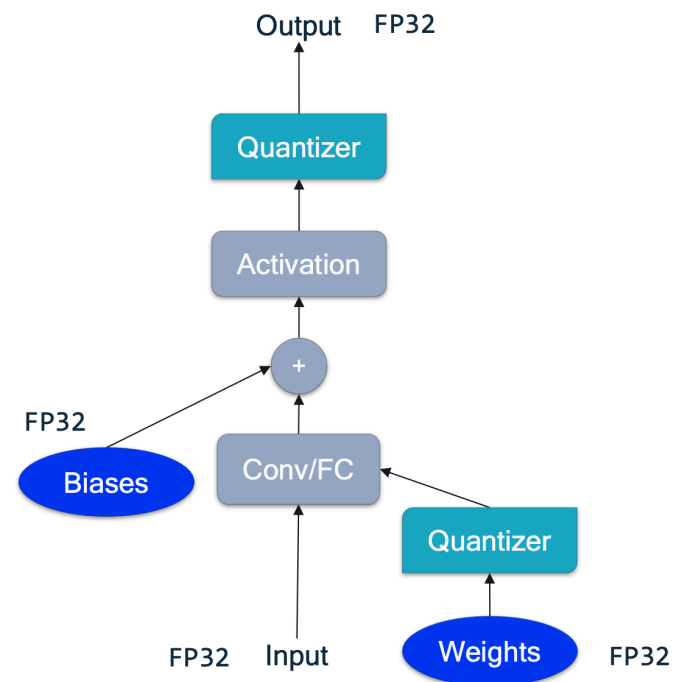


On-device vs. Simulated Quantization

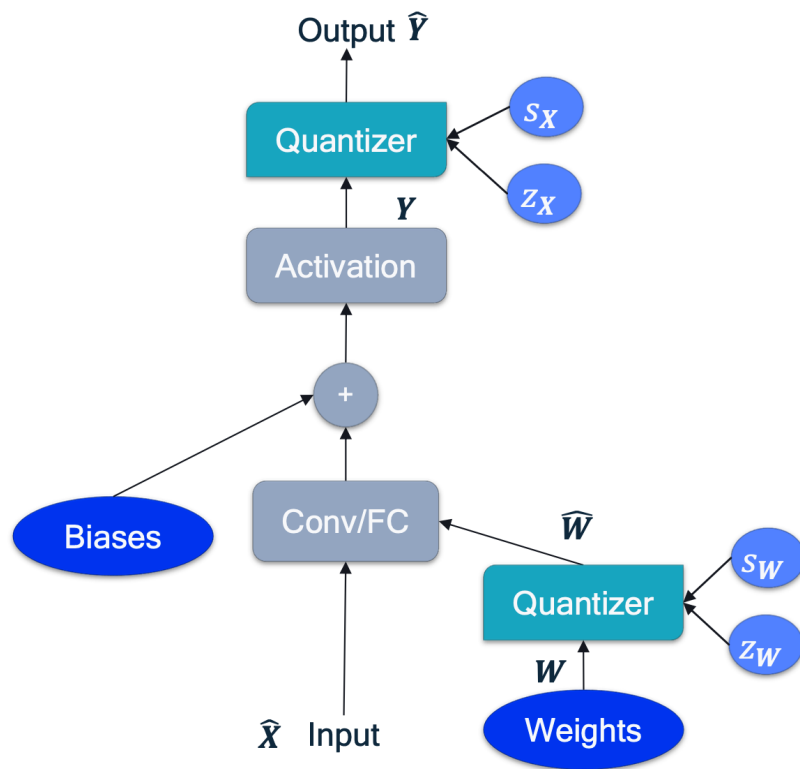
On-device fixed-point inference



Simulated quantized inference



Quantizers Operations

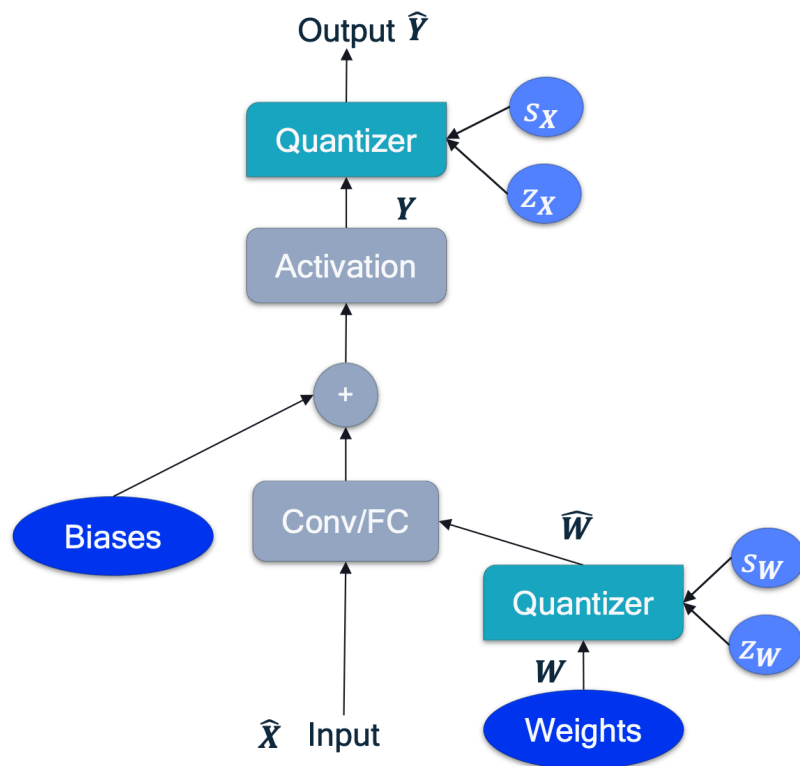


Assuming asymmetric quantization the quantization operation applied to input tensor X :

$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z)$$

Quantizers Operations



Assuming asymmetric quantization the quantization operation applied to input tensor X :

$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z)$$

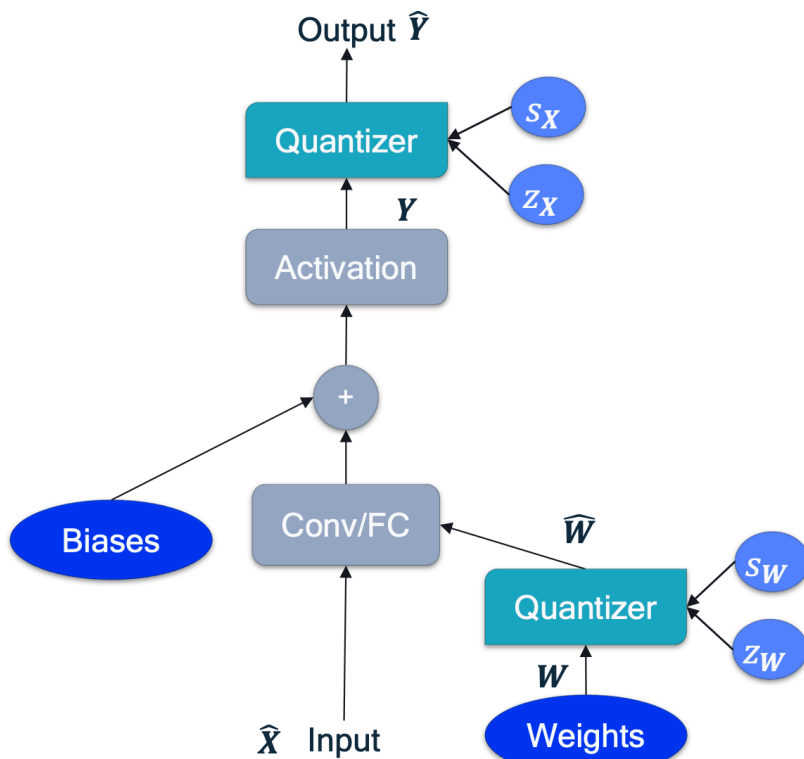
Example using $b = 4$:

$$X = \begin{pmatrix} 0.41 & 0.0 \\ 0.8 & -0.5 \end{pmatrix}$$

$$s = \frac{1}{15} = 0.067$$

$$z = \text{round} \left(\frac{0.5}{0.067} \right) = 8$$

Quantizers Operations



Assuming asymmetric quantization the quantization operation applied to input tensor X :

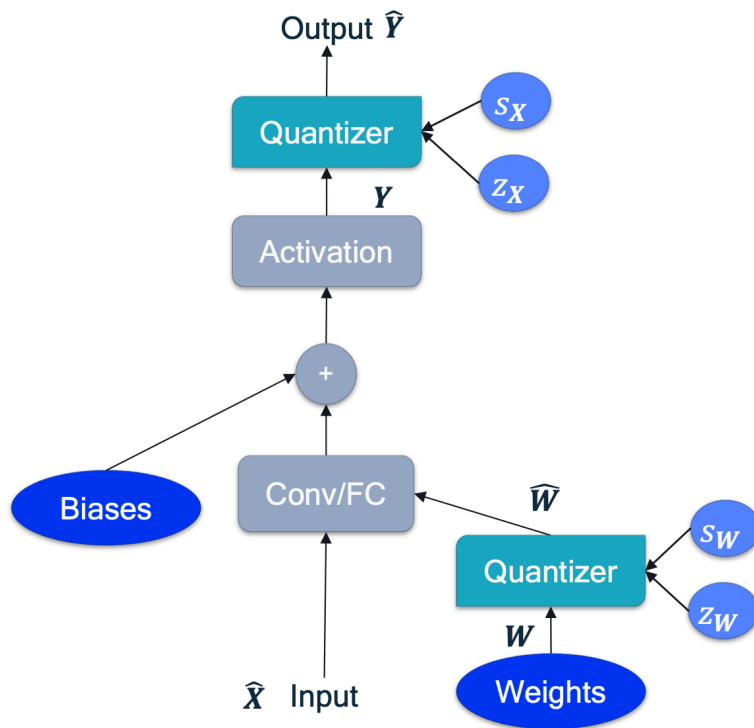
$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z) \quad \text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix}$$

Example using $b = 4$: $s = 0.067$ $z = 8$

$$\frac{X}{s} = \begin{pmatrix} 6.15 & 0.0 \\ 12 & -7.5 \end{pmatrix}$$

Quantizers Operations



Assuming asymmetric quantization the quantization operation applied to input tensor X :

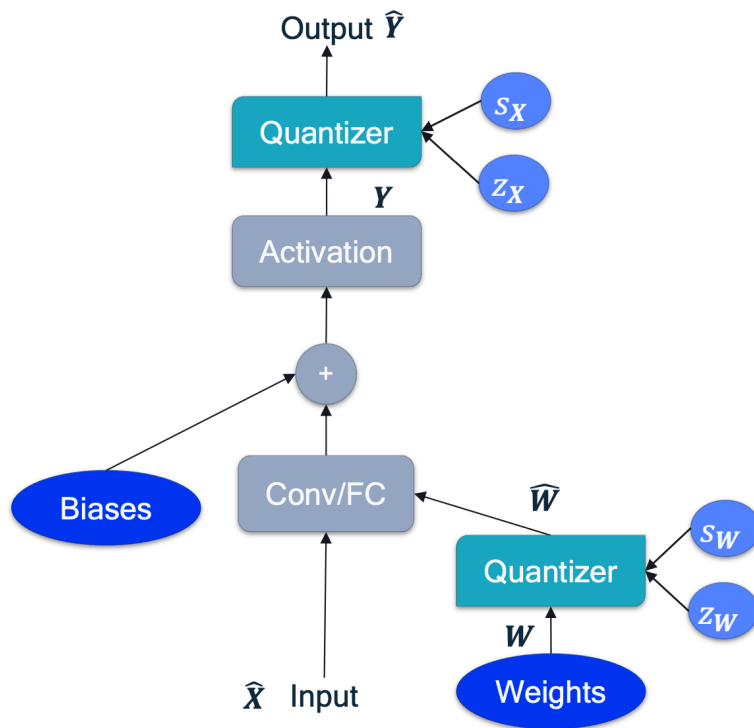
$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z) \quad \text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix}$$

Example using $b = 4$: $s = 0.067$ $z = 8$

$$\text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix}$$

Quantizers Operations



Assuming asymmetric quantization the quantization operation applied to input tensor X :

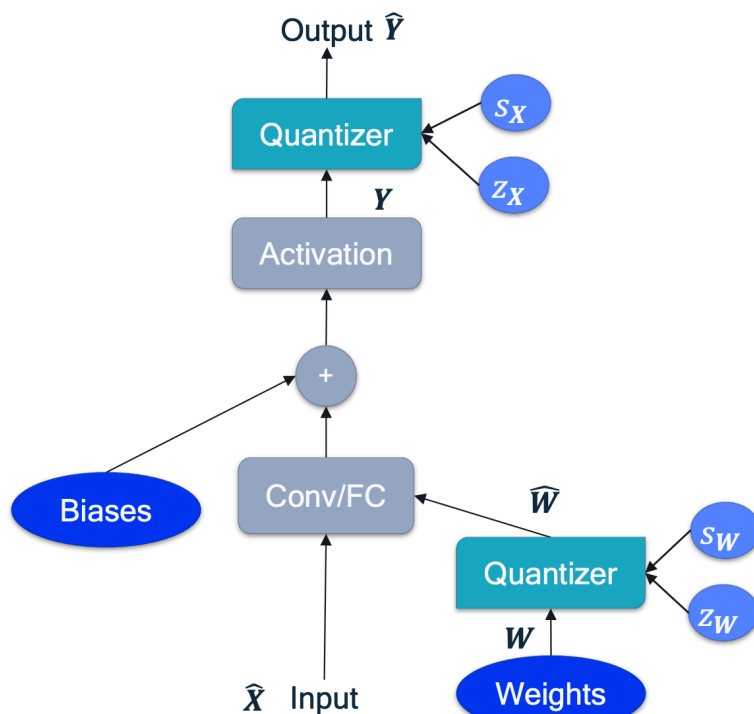
$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z) \quad \text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix}$$

Example using $b = 4$: $s = 0.067$ $z = 8$

$$\text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix} \xrightarrow{\text{clip}} \begin{pmatrix} 14 & 8 \\ \mathbf{15} & 0 \end{pmatrix}$$

Quantizers Operations



Assuming asymmetric quantization the quantization operation applied to input tensor X :

$$X_{\text{int}} = \text{clip} \left(\text{round} \left(\frac{X}{s} \right) + z, \min = 0, \max = 2^b - 1 \right)$$

$$\hat{X} = s (X_{\text{int}} - z) \quad \text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix}$$

Example using $b = 4$: $s = 0.067$ $z = 8$

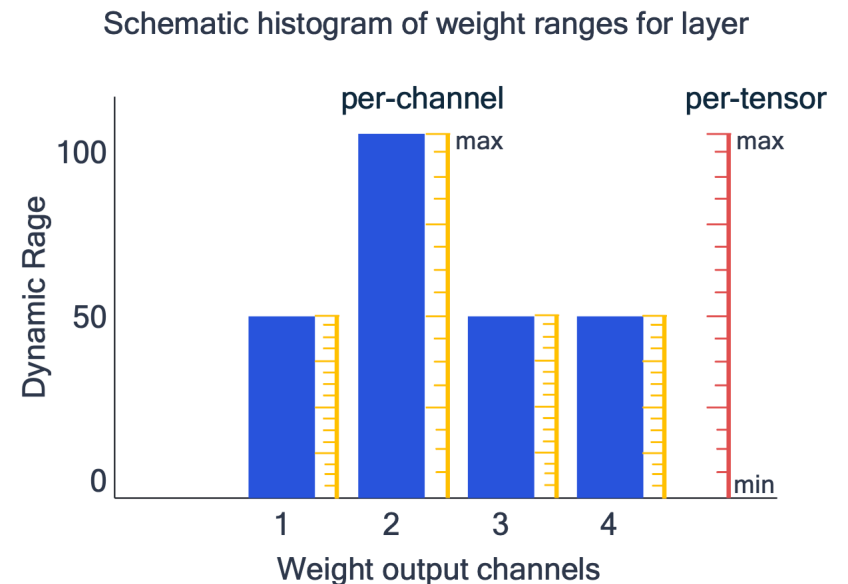
$$\text{round} \left(\frac{X}{s} \right) + z = \begin{pmatrix} 14 & 8 \\ 20 & 0 \end{pmatrix} \xrightarrow{\text{clip}} \begin{pmatrix} 14 & 8 \\ \mathbf{15} & 0 \end{pmatrix}$$

de-quantize $\downarrow$

$$X = \begin{pmatrix} 0.41 & 0.0 \\ \mathbf{0.8} & -0.5 \end{pmatrix} \quad \hat{X} = \begin{pmatrix} 0.4 & 0.0 \\ \mathbf{0.47} & -0.53 \end{pmatrix}$$

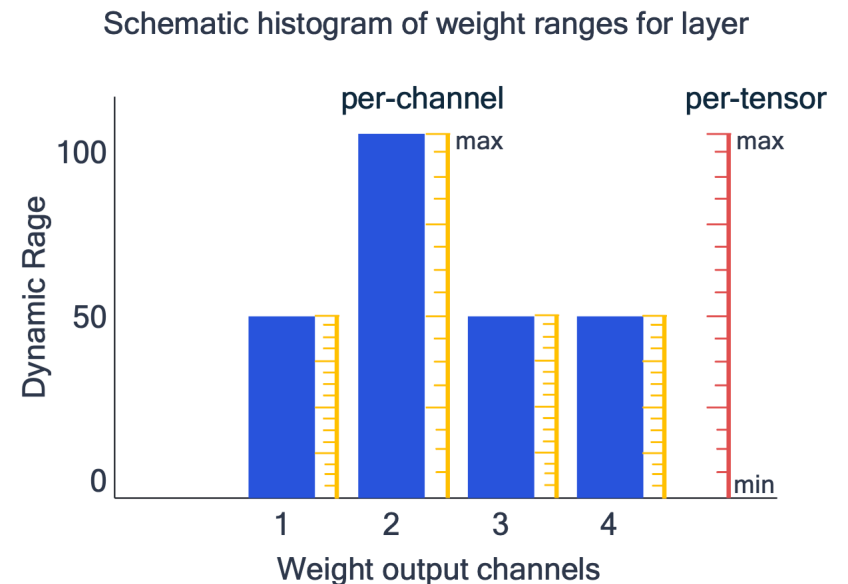
Per-channel vs Per-tensor Quantization of Weights

- **Per-tensor quantization** most supported by fixed-point accelerators
- **Per-channel quantization** better utilizes the quantization grid
- Per-channel quantization increasingly popular for weights
- Check for HW support

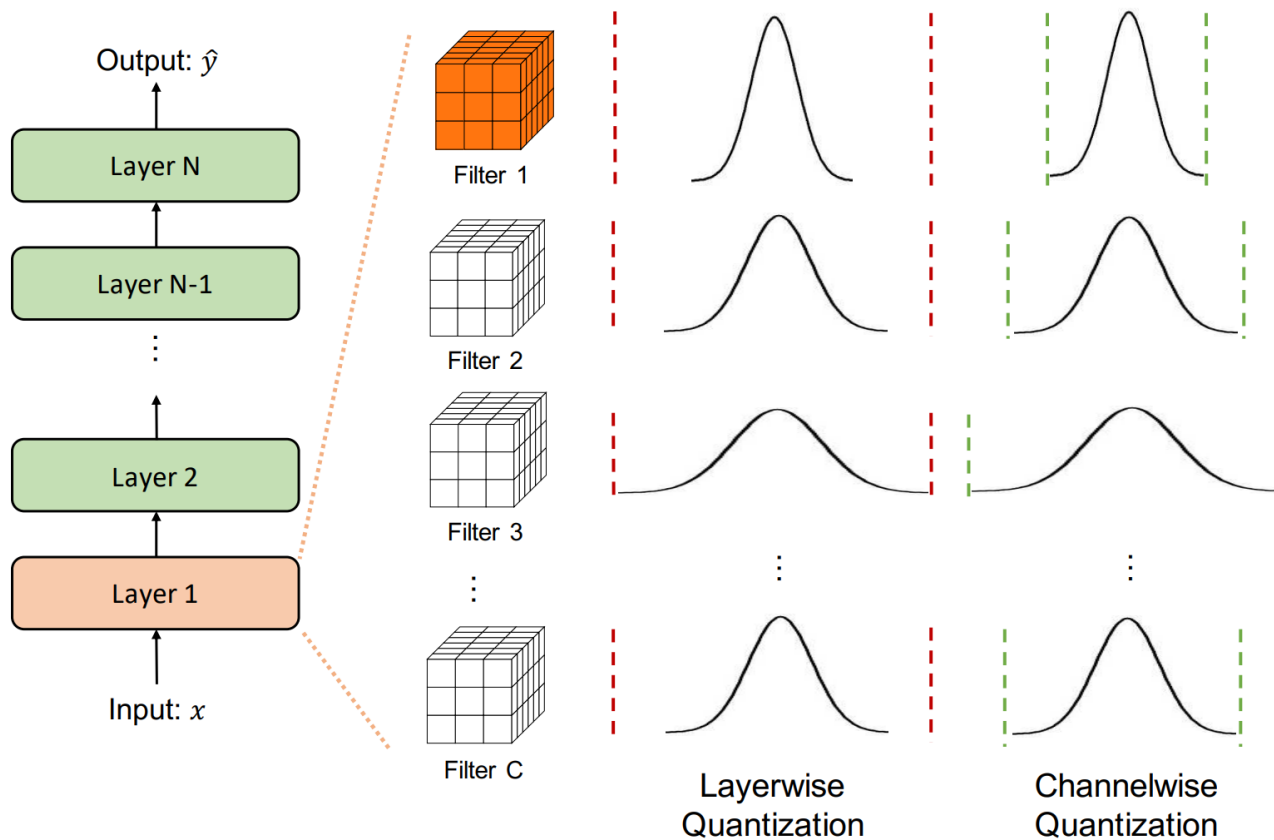


Per-channel vs Per-tensor Quantization of Weights

- **Per-tensor quantization** most supported by fixed-point accelerators
- **Per-channel quantization** better utilizes the quantization grid
- Per-channel quantization increasingly popular for weights
- Check for HW support



Per-tensor vs per-channel

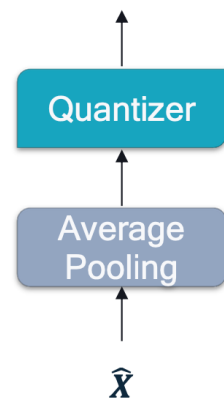
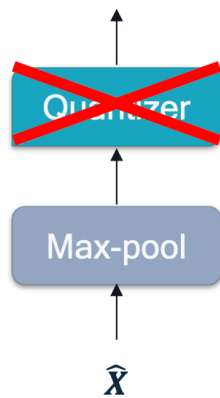


Per-channel is useful when channels have different ranges

Note: per-channel quantization does not require special hardware when applied to **weights**

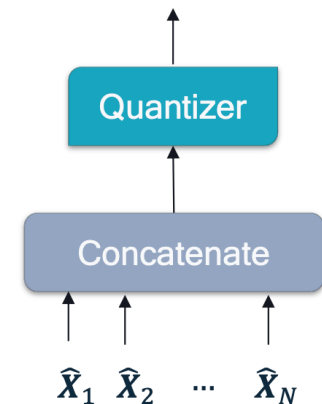
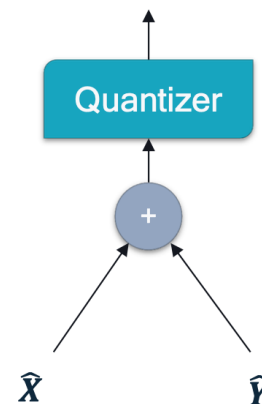
Image source: A Survey of Quantization Methods for Efficient Neural Network Inference – A. Gholami et al.

How to simulate quantization in common DL Layers



We can tie input
and output
quantizers

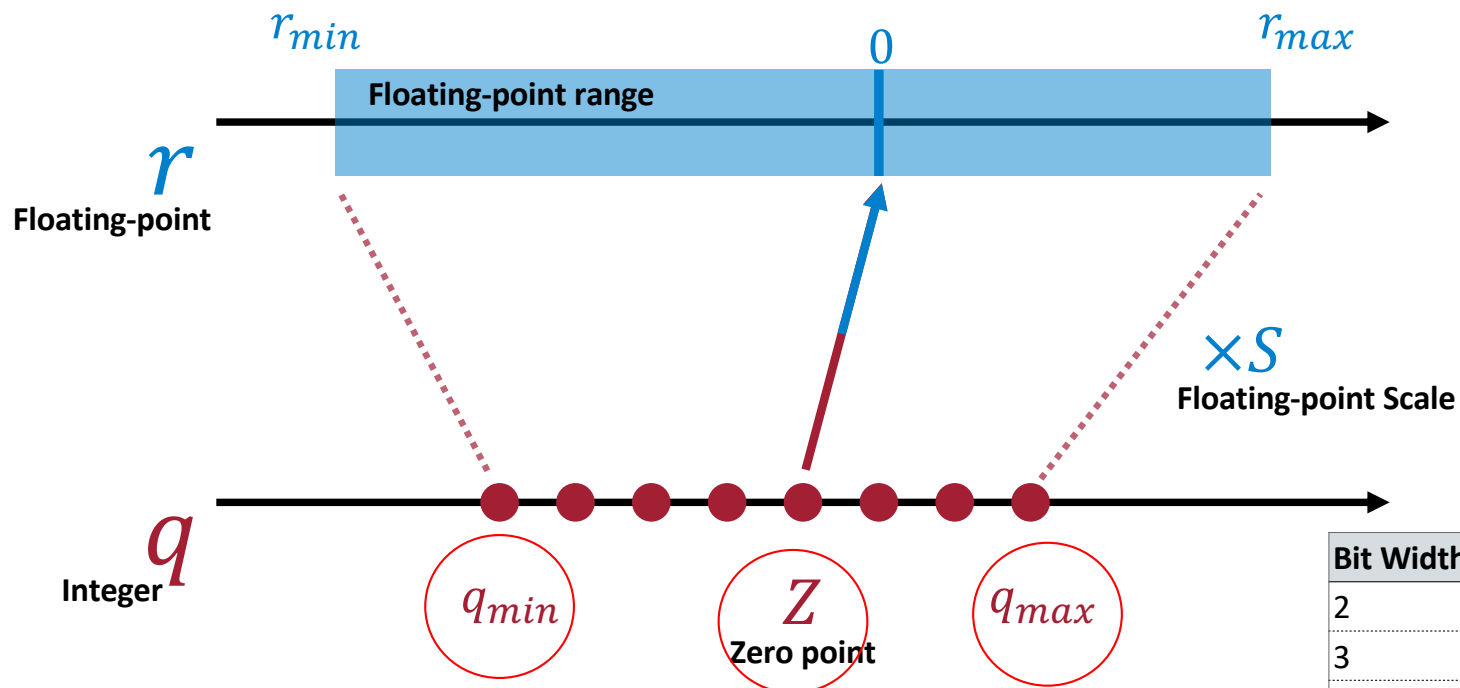
Elementwise Add



How to choose the right quantization Parameters?

Linear Quantization

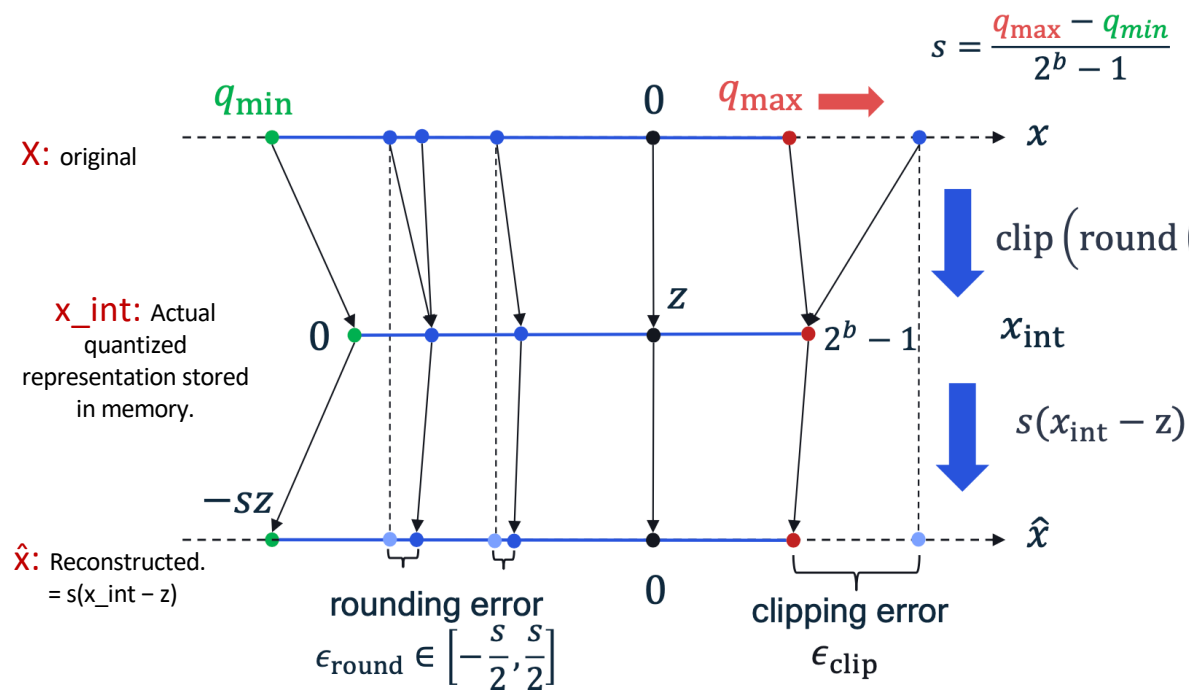
- An affine mapping of integers to real numbers $r = S(q - Z)$



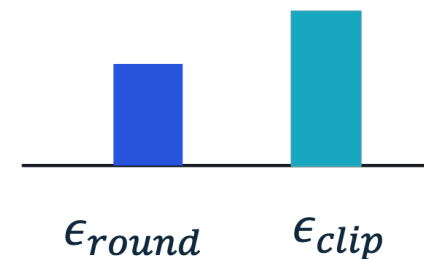
Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

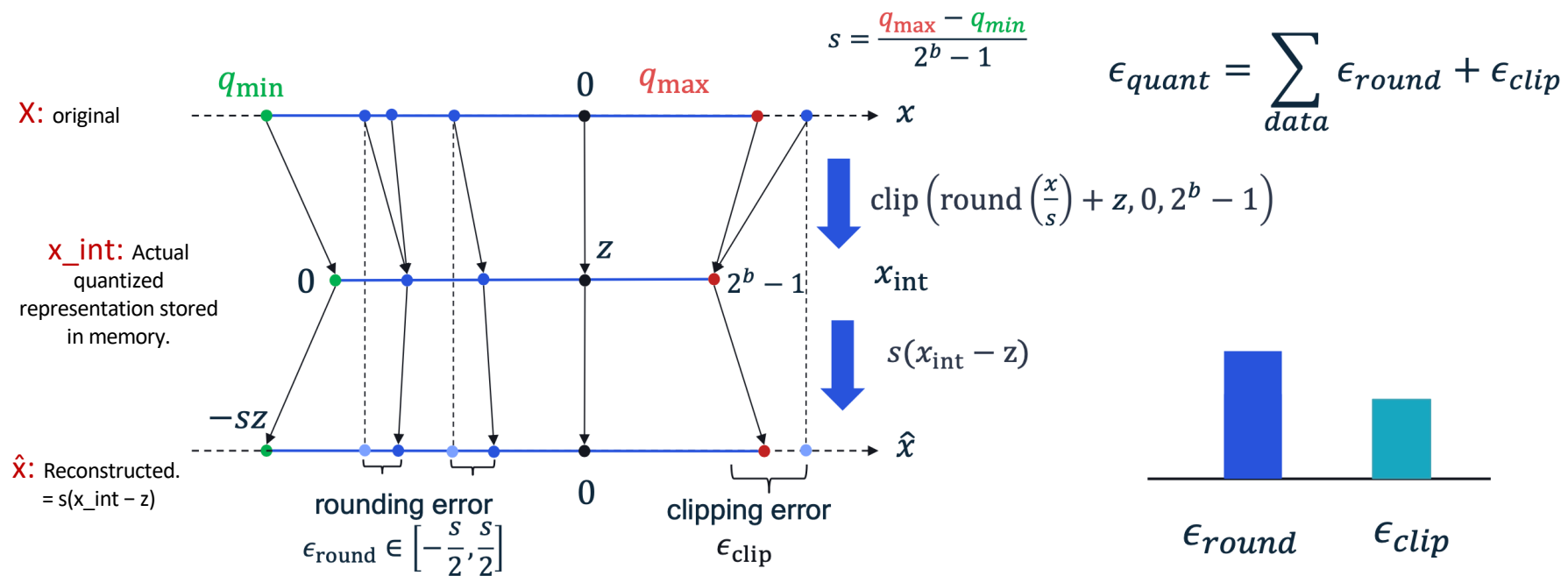
Sources of Quantization Error



$$\epsilon_{\text{quant}} = \sum_{\text{data}} \epsilon_{\text{round}} + \epsilon_{\text{clip}}$$



Sources of Quantization Error



Quantization Parameters Setting Methods

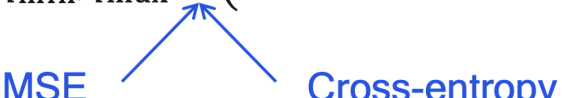
- Min-max range:

$$q_{\min} = \min X$$
$$q_{\max} = \max X$$

- Optimization-based methods:

$$\operatorname{argmin}_{q_{\min}, q_{\max}} \ell \left(X, \hat{X}(q_{\min}, q_{\max}) \right)$$

MSE Cross-entropy



- Batch-Norm Based [1]:

$$q_{\min} = \min (\boldsymbol{\beta} - \alpha \boldsymbol{\gamma})$$
$$q_{\max} = \max (\boldsymbol{\beta} + \alpha \boldsymbol{\gamma})$$

$$\text{BatchNorm}(\mathbf{z}_k)$$
$$= \boldsymbol{\gamma}_k \frac{\mathbf{z}_k - \boldsymbol{\mu}_k}{\sqrt{\boldsymbol{\sigma}_k + \epsilon}} + \boldsymbol{\beta}_k$$

[1] Nagel et al, 2019, Data-Free Quantization Through Weight Equalization and Bias Correction

Quantization Setting Methods Ablation Study

Model (FP32 Accuracy)	ResNet18 (69.68)		MobileNetV2 (71.72)	
Bit-width	A8	A6	A8	A6
Min-Max	69.60	68.19	70.96	64.58
MSE	69.59	67.84	71.35	67.55
MSE & X-entropy	69.60	68.91	71.36	68.85
BN ($\alpha = 6$)	69.54	68.73	71.32	71.32

Average ImageNet validation accuracy (%) over 5 seeds

When to Quantize?

What algorithm to choose to improve accuracy?

Two Approaches

Post-Training Quantization (PTQ)

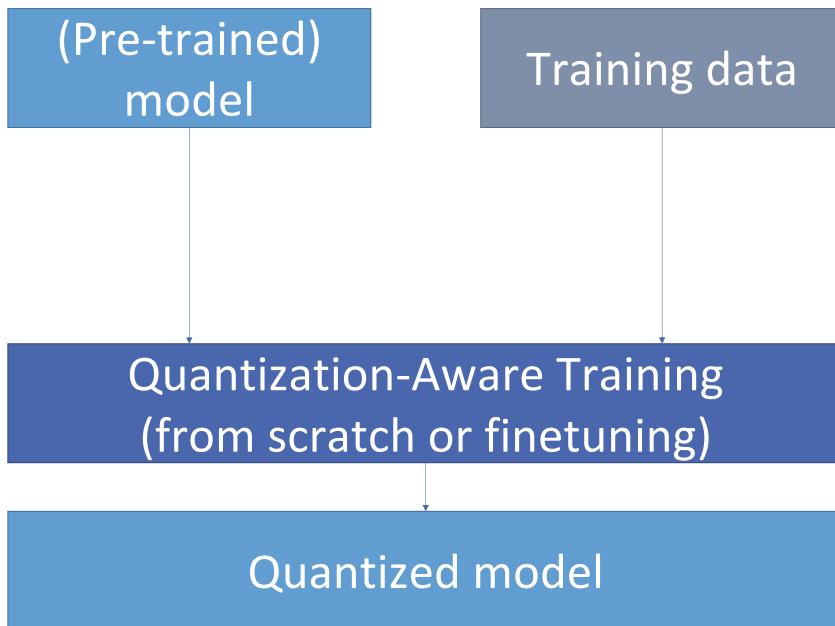
- ✓ Takes a pre-trained network and converts it to a fixed-point network without access to the training pipeline
- ✓ Data-free or small calibration set needed
- ✓ Use through single API call
- ✗ Lower accuracy at lower bit-widths

Quantization-Aware Training (QAT)

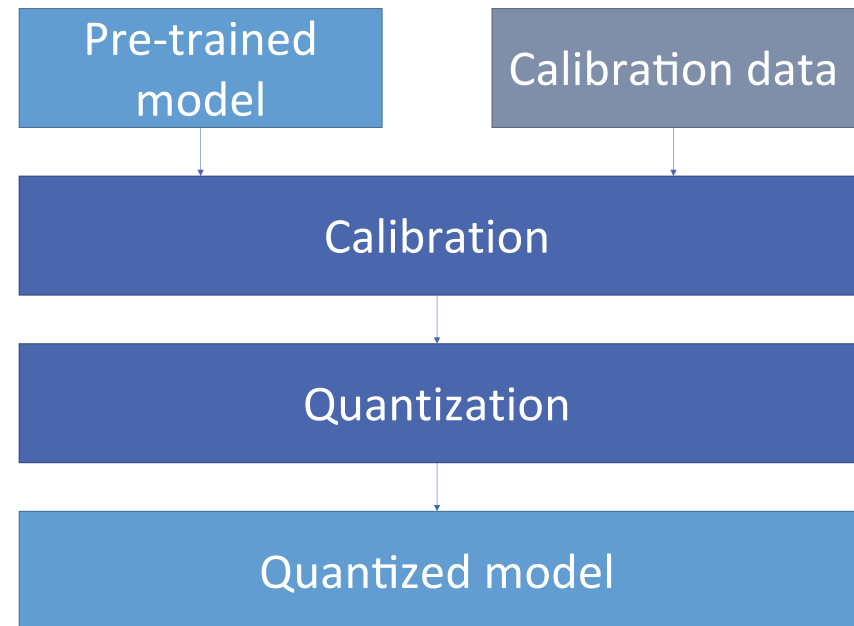
- ✗ Requires access to training pipeline and labelled data
- ✗ Longer training times
- ✗ Hyper-parameter tuning
- ✓ Achieves higher accuracy

When to quantize?

Quantization-Aware Training



Post-Training Quantization

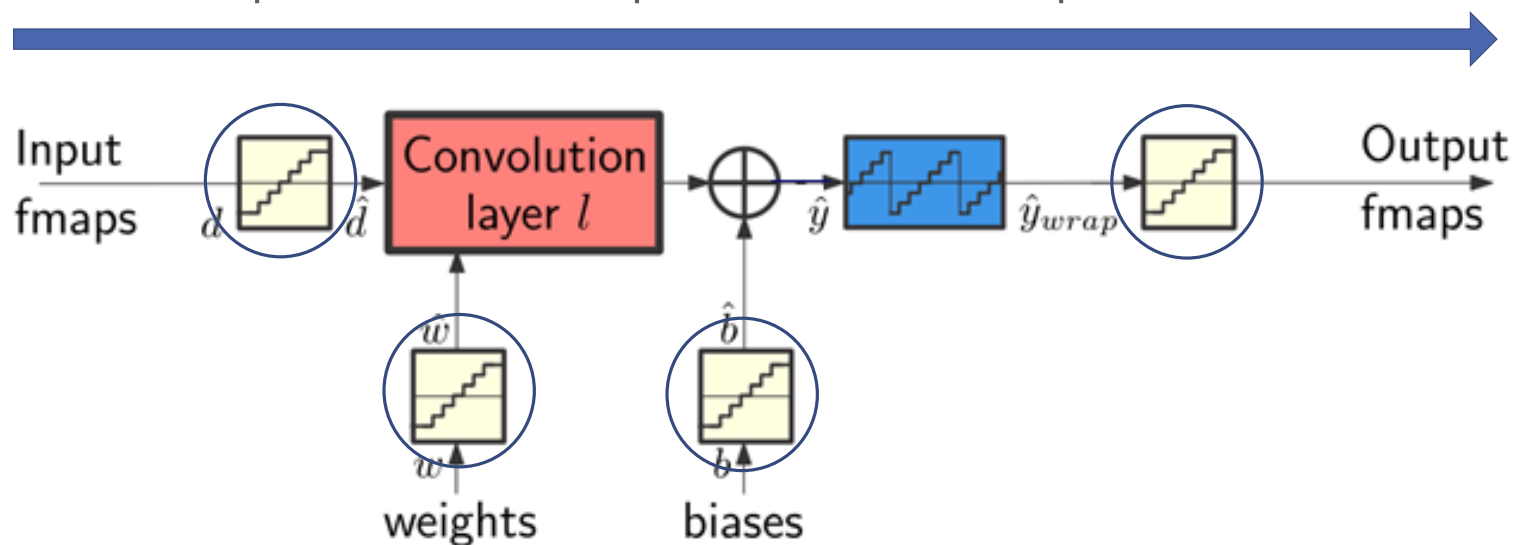


Post-Training Quantization

- Typical PTQ procedure:
 1. Range analysis using calibration data (subset of training data)
 - To determine step size for activations
 - Step size for weights can be determined without any data
 2. Check if quality of final solution is sufficient. If not, do some finetuning (using entire training data)

Quantization-Aware Training

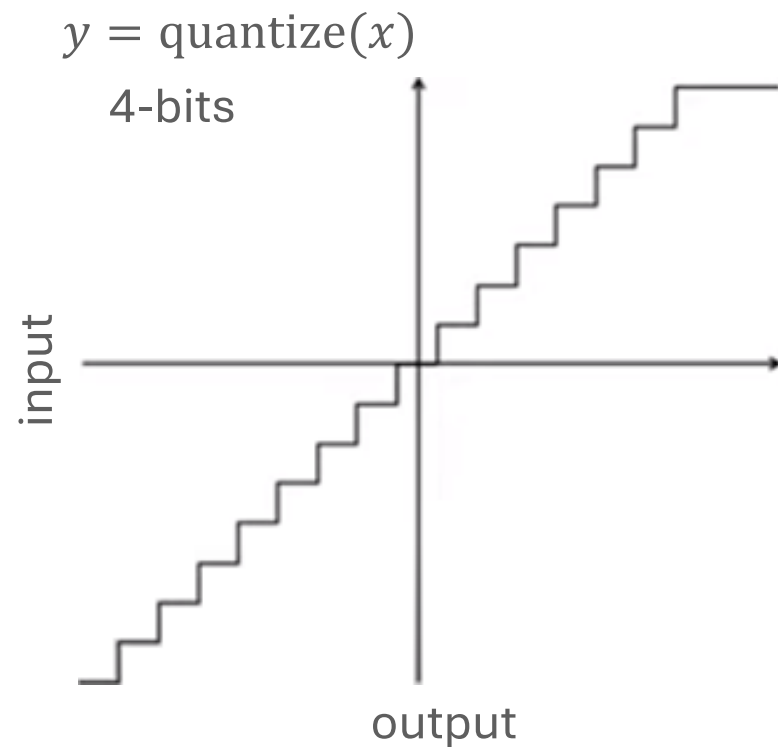
Forward pass: simulated quantization to compute loss function



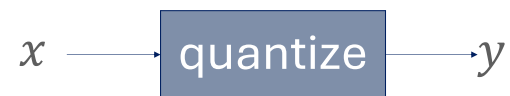
Backward pass: how to back propagate through the quantizers?

Image source: Quantization of deep neural networks for accumulator-constrained processors – B. de Bruin et al.

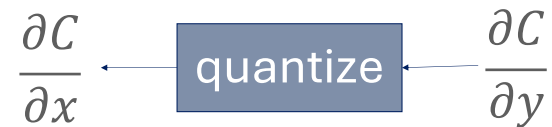
Quantizers: how to backpropagate?



Forward pass



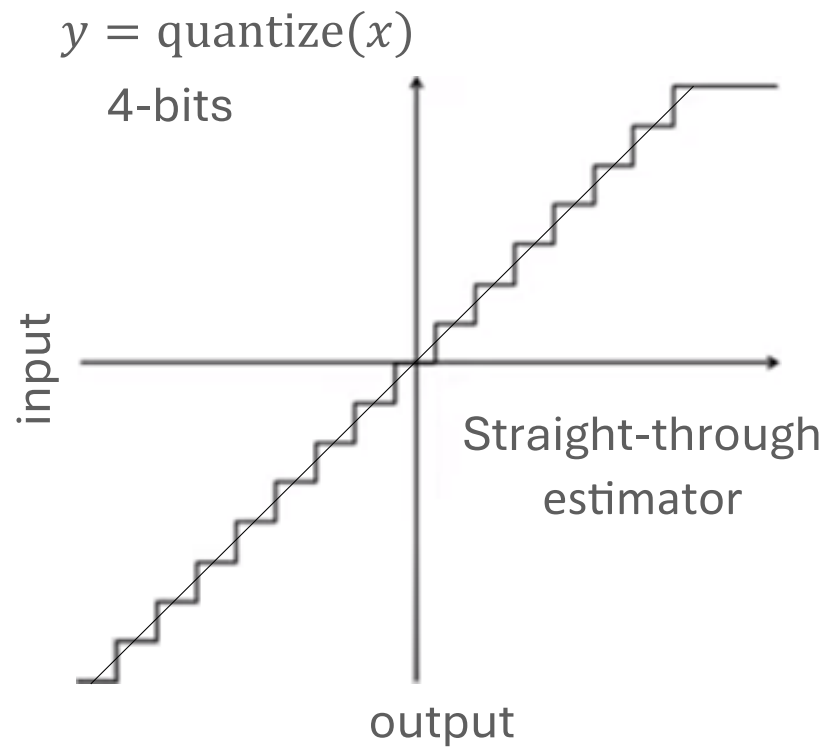
Backward pass



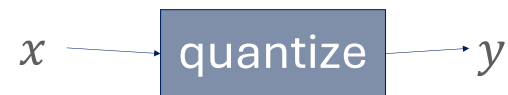
$$\frac{\partial \mathcal{C}}{\partial x} = \frac{\partial \mathcal{C}}{\partial y} \left(\frac{\partial y}{\partial x} \right)$$

?

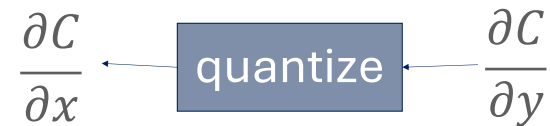
Quantizers: how to backpropagate?



Forward pass



Backward pass



$$\frac{\partial C}{\partial x} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial x}$$

➡ $\frac{\partial C}{\partial y} 1$

(STE) ([Hinton et al., 2012](#), [Bengio, 2013](#))

Quantization Approaches - Summary

Approach	Training time	Quantization time	Quality of results
Post-Training Quantization	None	1 forward pass	Low
Post-Training Quantization with fine-tuning	Short	Multiple forward & backward passes	Medium
Quantization-Aware Training	Long	Entire training	High

Notable Quantization Algorithms

Mixed-Precision Quantization

Core Concept

Problem: Not all layers equally important

Solution: Assign different bit-widths to different layers

HAQ: Hardware-aware with latency/energy constraints

Hessian-based: Use second-order info for sensitivity

Layer sensitivity: Measure per-layer quantization impact

Typical Configuration

Critical layers: 8-bit or 4/8-bit mixed

Less sensitive layers: 4-bit or even 2-bit

Output layers: Often kept at higher precision

Optimal balance between size and accuracy

GPTQ: GPU-Optimized Quantization

What is GPTQ?

- Post-training quantization method for GPU inference
- Based on Optimal Brain Quantization (OBQ)
- Layer-wise quantization using Hessian matrix
- Key innovations: Cholesky reformulation, lazy batch updates
- Arbitrary order processing for efficiency

Published as a conference paper at ICLR 2023

GPTQ: ACCURATE POST-TRAINING QUANTIZATION FOR GENERATIVE PRE-TRAINED TRANSFORMERS

Elias Frantar*
IST Austria

Salih Ashkboos
ETH Zurich

Torsten Hoefler
ETH Zurich

Dan Alistarh
IST Austria & NeuralMagic

ABSTRACT

Generative Pre-trained Transformer models, known as GPT or OPT, set themselves apart through breakthrough performance across complex language modelling tasks, but also by their extremely high computational and storage costs. Specifically, due to their massive size, even inference for large, highly-accurate GPT models may require multiple performant GPUs, which limits the usability of such models. While there is emerging work on relieving this pressure via model compression, the applicability and performance of existing compression techniques is limited by the scale and complexity of GPT models. In this paper, we address this challenge, and propose GPTQ, a new one-shot weight quantization method based on approximate second-order information, that is both highly-accurate and highly-efficient. Specifically, GPTQ can quantize GPT models with 175 billion parameters in approximately four GPU hours, reducing the bitwidth down to 3 or 4 bits per weight, with negligible accuracy degradation relative to the uncompressed baseline. Our method more than doubles the compression gains relative to previously-proposed one-shot quantization methods, preserving accuracy, allowing us for the first time to execute a 175 billion-parameter model inside a single GPU for generative inference. Moreover, we also show that our method can still provide reasonable accuracy in the *extreme quantization* regime, in which weights are quantized to 2-bit or even *ternary* quantization levels. We show experimentally that these improvements can be leveraged for end-to-end inference speedups over FP16, of around 3.25x when using high-end GPUs (NVIDIA A100) and 4.5x when using more cost-effective ones (NVIDIA A6000). The implementation is available at <https://github.com/IST-DASLab/gptq>.

1 INTRODUCTION

ICLR 2023: <https://arxiv.org/abs/2210.17323>

GPTQ: GPU-Optimized Quantization

Performance & Limitations

- Typically, 4-bit quantization (INT4/FP16 mixed)
- 5x faster on GPU
- Real-time dequantization during inference
- Limitations: Requires calibration dataset
- Quantization process can be time-consuming

Published as a conference paper at ICLR 2023

GPTQ: ACCURATE POST-TRAINING QUANTIZATION FOR GENERATIVE PRE-TRAINED TRANSFORMERS

Elias Frantar*
IST Austria

Saleh Ashkboos
ETH Zurich

Torsten Hoefler
ETH Zurich

Dan Alistarh
IST Austria & NeuralMagic

ABSTRACT

Generative Pre-trained Transformer models, known as GPT or OPT, set themselves apart through breakthrough performance across complex language modelling tasks, but also by their extremely high computational and storage costs. Specifically, due to their massive size, even inference for large, highly-accurate GPT models may require multiple performant GPUs, which limits the usability of such models. While there is emerging work on relieving this pressure via model compression, the applicability and performance of existing compression techniques is limited by the scale and complexity of GPT models. In this paper, we address this challenge, and propose GPTQ, a new one-shot weight quantization method based on approximate second-order information, that is both highly-accurate and highly-efficient. Specifically, GPTQ can quantize GPT models with 175 billion parameters in approximately four GPU hours, reducing the bitwidth down to 3 or 4 bits per weight, with negligible accuracy degradation relative to the uncompressed baseline. Our method more than doubles the compression gains relative to previously-proposed one-shot quantization methods, preserving accuracy, allowing us for the first time to execute a 175 billion-parameter model inside a single GPU for generative inference. Moreover, we also show that our method can still provide reasonable accuracy in the *extreme quantization* regime, in which weights are quantized to 2-bit or even *ternary* quantization levels. We show experimentally that these improvements can be leveraged for end-to-end inference speedups over FP16, of around 3.25x when using high-end GPUs (NVIDIA A100) and 4.5x when using more cost-effective ones (NVIDIA A6000). The implementation is available at <https://github.com/IST-DASLab/gptq>.

1 INTRODUCTION

ICLR 2023: <https://arxiv.org/abs/2210.17323>

Layer-Wise Quantization

- Equation:

- $\arg \min_{\widetilde{W}_\ell} ||W_\ell X_\ell - \widetilde{W}_\ell X_\ell||_2^2$

- Explanation:

- This equation defines layer-wise quantization as an optimization problem.
 - W_ℓ : full-precision weight matrix of layer ℓ
 - X_ℓ : input activations for that layer
 - $\widetilde{W}_\ell$: quantized weights chosen to minimize reconstruction error
- The goal is to find quantized weights that make the output of the layer as close as possible to the original full-precision output.

GPTQ: One-Shot Quantization Using Second-Order

- GPTQ adapts second-order ideas for post-training 4-bit quantization
- Workflow summary:
 - Collect activations X via calibration set.
 - Solve for quantized weights $\hat{W}$ that minimize $\|WX - \hat{W}X\|^2 + \text{second-order regularization}$.
 - Use GPU-friendly kernel for final inference.
- Similar to OBD: uses curvature to guide weight modification
- Enables accurate 3–4 bit quantization for large LLMs
- **GPTQ also uses Hessian/second-order info, but:**
 - it uses a **cheaper approximation**
 - it updates weights in a **fixed order**
 - it avoids expensive re-optimization per weight

GPTQ Performance

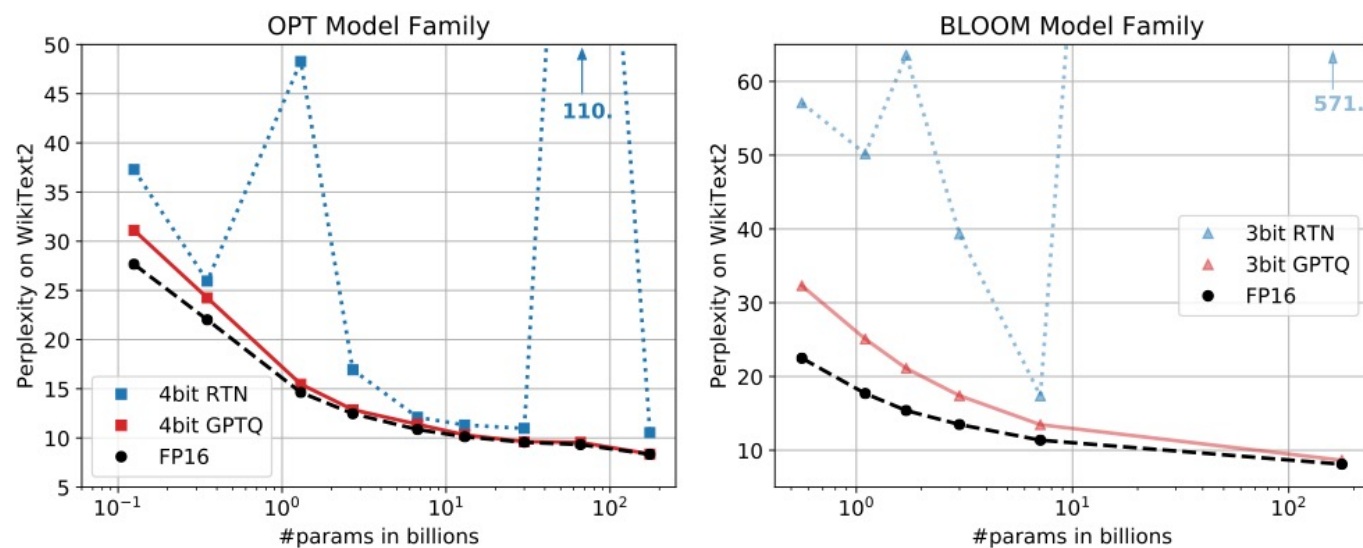


Figure 1: Quantizing OPT models to 4 and BLOOM models to 3 bit precision, comparing GPTQ with the FP16 baseline and round-to-nearest (RTN) (Yao et al., 2022; Dettmers et al., 2022).

AWQ: Activation-Aware Weight Quantization

Core Concept

Not all weights are equally important

Protects salient weights based on activation distributions

Salient weights (<1% of total) skip quantization

Process: Run calibration → Collect activation statistics

Identify salient weights → Apply selective quantization

Performance Advantages

Often outperforms GPTQ in speed

Better preserves generalization ability

No backpropagation required (faster quantization)

Excellent for instruction-tuned and multi-modal LLMs

Less prone to overfitting calibration set

Other Notable Quantization Methods

Additional Techniques

GGUF/GGML: CPU-optimized format, Apple Silicon support

SmoothQuant: Addresses activation outliers, weight-activation trade-off

QLoRA: 4-bit NF4 + LoRA adapters for fine-tuning

Enables fine-tuning on consumer GPUs

Used via llama.cpp (2-8 bit precision)

Hardware Support for Quantization

GPU Hardware Support: The FP8 Revolution

NVIDIA Hopper (H100)

Native FP8 support introduced in 2022

Two formats: E4M3 (precision) & E5M2 (range)

2x compute speedup vs FP16

2x memory reduction

Peak: 1513 TFLOPS (FP8) vs 756.5 (FP16)

Why FP8 > INT8 for LLMs

Better outlier handling: Adaptive scale via exponent

Wider dynamic range: Critical for attention mechanisms

Easier quantization: Less accuracy loss than INT8

Training capable: Can be used for full training, not just inference

GPU Support Across Architectures

NVIDIA GPUs

Hopper (H100, 2022): FP8, INT8 native support

Blackwell (2024): FP8, FP6, FP4 (NVFP4)

Ada Lovelace (RTX 40xx): FP8 support

Ampere (A100): INT8 primarily

Industry trend: Convergence on E4M3/E5M2 FP8 formats

Intel & AMD

Intel Gaudi 2 HPU: Full FP8 support with matrix units

Higher throughput-to-power efficiency in some workloads

AMD MI300 series: FP8 matrix core support (E5M2)

Industry FP8 standardization across vendors

NVIDIA Support for Quantization

- Software Support and Tooling



NVIDIA TensorRT: This is the primary platform for high-performance deep learning inference and natively supports quantization. It enables the deployment of models in low-precision formats such as INT8, FP8, INT4, and FP4.



NVIDIA Model Optimizer (ModelOpt): This library, integrated into frameworks like NeMo, offers advanced post-training quantization (PTQ) techniques, including SmoothQuant and Activation-Aware Weight Quantization (AWQ).



PyTorch Quantization Toolkit: This toolkit facilitates quantization-aware training (QAT) within the PyTorch framework, allowing developers to simulate lower precision during training to minimize accuracy loss.



Framework Integration: NVIDIA's tools integrate with popular AI frameworks like PyTorch and TensorFlow, making it easier to apply quantization techniques across different model architectures.

NVIDIA Hardware Acceleration for Quantization



GPU	Architecture	Release Year	Tensor Core Generation	Quantization Hardware Support	Primary Quantization Use Case
H100 / H200	Hopper	2022(H100)	4th Gen	Native FP8 support, Transformer Engine (dynamic FP8/FP16/BF16 scaling), High HBM3/HBM3e bandwidth for large models.	Large-scale AI Training & Inference (FP8-based mixed precision).
Blackwell GPUs (e.g., B200, consumer RTX 5090)	Blackwell	2025 (Consumer)	5th Gen	Native FP4, FP6, and FP8 support (NVFP4 format), Second-Gen Transformer Engine for optimized low-bit LLMs.	Next-generation, extreme efficiency LLM Inference and 4-bit training.
RTX 6000 Ada	Ada Lovelace	2022	4th Gen	Native FP8 support, Transformer Engine , 48GB VRAM with ECC for high reliability.	High-end Workstations and professional fine-tuning with FP8.
RTX 4090	Ada Lovelace	2022	4th Gen	Native FP8 support, Transformer Engine (hardware support for FP8 in Tensor Cores), 24GB VRAM.	Consumer/Prosumer LLM Projects , inference, and small-to-medium FP8 fine-tuning.
L40S	Ada Lovelace	2022	4th Gen	Native FP8 support, Transformer Engine , 48GB VRAM, optimized for cloud deployments.	Cloud/On-prem Inference deployments where efficiency and capacity are key.

LLM Quantization

Quantization in Modern LLMs

LLaMA Family

LLaMA 2 (7B, 13B, 70B) widely quantized to 4-bit

LLaMA 3.3 (70B) and LLaMA 4
(Scout/Maverick, Apr 2025)

Common: 4-bit weights, 16-bit activations

GGUF popular for CPU/Apple Silicon

Examples: Mistral 7B FP32 ~28GB → 2-bit
~2.5GB

Mistral 7B

Outperforms LLaMA 2 13B despite smaller size

Uses Grouped-Query Attention (GQA)

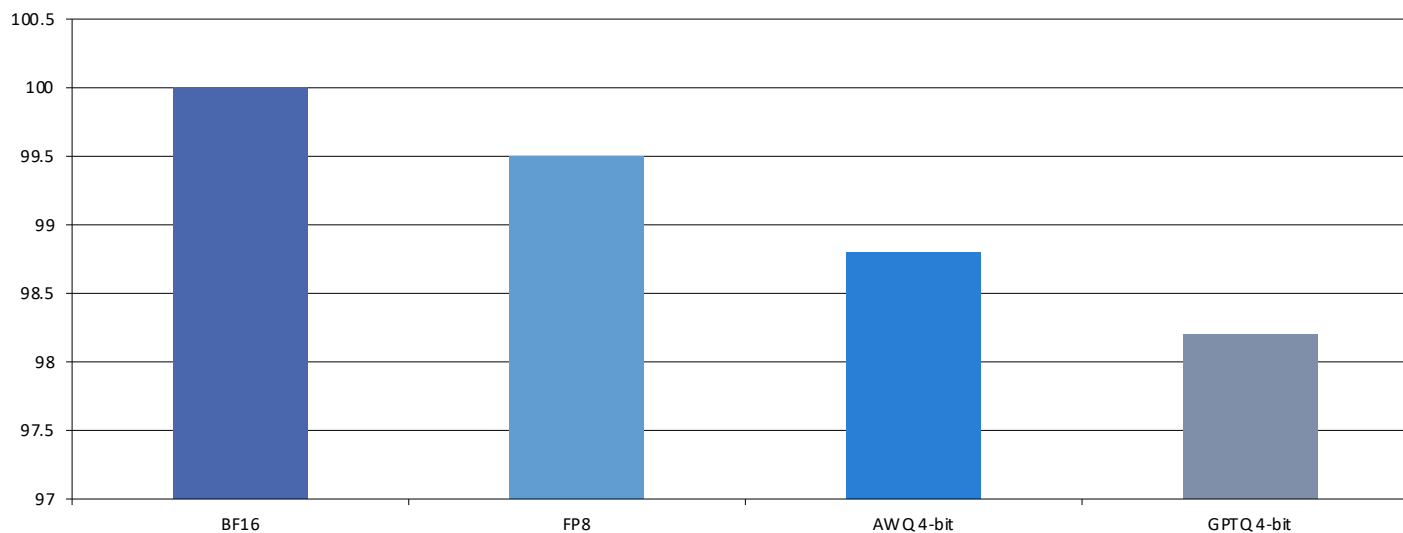
Sliding Window Attention for efficiency

Typical deployment: 2-bit to 8-bit GGUF

Trade-offs: 8-bit <1% loss, 4-bit good with
AWQ/GPTQ

Quantization Performance Comparison

LLaMA 3.1 70B Benchmark Results



Key Findings: AWQ outperforms GPTQ • FP8 offers best accuracy-performance trade-off • Smaller models more sensitive

Quantization in Production LLM Systems

Current Industry Practices

OpenAI/GPT: 8-bit inference widely reported

Meta LLaMA: Official FP8 + community 4-bit

Cloud providers: AWS Bedrock, Mistral
Compute

vLLM: Optimized serving with AWQ/GPTQ
support

Typical Production Stack

Training: BF16 or FP8 (Hopper+)

Deployment: FP8 (new) or INT8 (older
hardware)

Edge/Local: 4-bit (GPTQ/AWQ/GGUF)

Memory: 7B model FP16 ~14GB, FP8 ~7GB, 4-bit
~3.5GB

Advanced Techniques & Future Trends

Emerging Methods (2025-2026)

QuaRot: Outlier-free 4-bit via rotation (NeurIPS 2024)

SpinQuant: Learned rotations for better distribution

Task-Aware Quantization: Task-specific precision allocation

LLM-FP4: 4-bit floating-point for transformers

DeepSeek-V3: FP8 mixed-precision full training at scale (2025)

Future Directions

Sub-4-bit quantization: FP4, FP6 on Blackwell

Training in low precision: Full FP8/INT8 training

Hardware-software co-design approaches

Dynamic quantization: Runtime adaptation

Practical Considerations for Deployment

Deployment Strategies

GPU: Hopper/Blackwell use FP8, older GPUs use GPTQ/AWQ 4-bit

CPU/Apple Silicon: GGUF format with llama.cpp (2-8 bit)

Edge Devices: INT8 or 4-bit, QLoRA for fine-tuning

Tools: AutoGPTQ, AutoAWQ, bitsandbytes, TensorRT-LLM, vLLM

Calibration: 128-1000 samples typically sufficient

Hands-on Example: Quantizing Mistral 7B

```
# Install dependencies
pip install auto-gptq transformers

# Load and quantize
from transformers import AutoModelForCausalLM, GPTQConfig

quantization_config = GPTQConfig(
    bits=4, group_size=128, dataset="c4"
)

model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.2",
    quantization_config=quantization_config
)

# Save quantized model
model.save_pretrained("./mistral-7b-gptq")
```

Results

Original: ~14GB (FP16)

GPTQ 4-bit: ~3.8GB

Speed: 3-5x faster

Accuracy loss: <2%

Pre-quantized models available on
Hugging Face

Challenges & Limitations

Current Challenges

Accuracy degradation from activation outliers

Calibration dataset quality dependency

Hardware limitations (FP8 only on newest GPUs)

Task-specific performance variation

Memory bandwidth still a bottleneck

Known Issues

GPTQ: Can overfit calibration data, slower quantization

AWQ: More complex implementation, limited architecture support

FP8: Limited hardware availability, conversion overhead for small batches

Training stability requires careful tuning

Measuring Quantization Quality

Key Metrics

Perplexity: Prediction quality (lower is better)

Task-specific accuracy: MMLU, HumanEval,
GSM8K

Inference performance: Latency, throughput,
memory

Hardware utilization: GPU usage, power
consumption

KL divergence: Per-token probability
comparison

Best Practices

Always compare on your specific use case

Measure both accuracy AND speed

Consider total cost of ownership (TCO)

Test with realistic workloads

Use lm-evaluation-harness for benchmarking

Key Takeaways

1. Quantization is Essential: Enables deployment with 2-4x memory reduction and 2-5x speed improvement
2. Multiple Approaches Available: FP8 (best trade-off), GPTQ/AWQ (4-bit GPU), GGUF (CPU), QLoRA (fine-tuning)
3. Hardware Acceleration Critical: Modern GPUs (Hopper, Blackwell) have native FP8 support
4. Practical Considerations: Choose method based on target, calibration matters, measure on your tasks
5. Active Research Area: New methods emerging (2024-2025), pushing toward 2-bit and training in reduced precision

Quantization Scheme Decision Guide

For each tensor type: choose the right numeric format, granularity, and symmetry

Weights	Activations	Gradients and Training
Symmetry * Symmetric Z=0 (default) Weights ~zero-centered after training Granularity * Per-channel (8-bit, default) * Per-group 128 (4-bit LLMs) Format by precision * 8-bit: INT8 symmetric * 4-bit: NF4 or INT4 per-group * Training: FP8 E4M3 <i>Z=0 eliminates zero-point arithmetic => hardware efficient MAC units</i>	Symmetry * Asymmetric Z!=0 (default) ReLU: always non-negative Sigmoid/Softmax: range [0,1] Granularity * Per-tensor (CNNs, vision) * Per-token (LLM decode) * Dynamic (varies per input) Format by precision * 8-bit: UINT8 (post-ReLU) * W4A16: keep acts in FP16 * Training fwd: FP8 E4M3 <i>Calibration data required -- distribution varies per input</i>	Symmetry * Asymmetric (gradients skewed) Need high dynamic range Format by role * Fwd pass: FP8 E4M3 (precision) * Bwd pass: FP8 E5M2 (range) * Weight update: BF16 / FP32 * Adam m,v states: FP32 Real example (DeepSeek-V3) * Linear fwd/bwd: FP8 * Attention, norm: BF16 <i>E5M2: more range for gradient explosion; E4M3: more precision for forward pass activations</i>

Golden Rule: Symmetric weights (Z=0) + Asymmetric activations = best hardware efficiency and accuracy tradeoff

Algorithm Selection for Modern Workloads

Workload	Hardware Target	Algorithm / Method	Format	Accuracy Loss	Tool / Library
LLM Server Inference High throughput, batch	H100 / H200 / B200 Hopper / Blackwell	FP8 + Transformer Engine Dynamic scaling, PTQ-free	W8A8 FP8 E4M3 / E5M2	< 0.5%	TensorRT-LLM, vLLM
LLM Server Inference Older hardware	A100 / V100 / RTX 40xx Ampere / Ada Lovelace	AWQ or GPTQ (PTQ) AWQ preferred for accuracy	W4A16 INT4	< 1%	AutoAWQ, AutoGPTQ, vLLM
LLM Local / On-Device CPU, Apple Silicon, edge	CPU / Apple M-series ARM / x86 / NPU	GGUF / GGML (llama.cpp) Non-linear block quantization	Q4_K_M (default) Q5/Q8 for accuracy	1-3%	Ollama, LM Studio
Fine-tuning Large Models Limited GPU memory	Any GPU (24-80 GB) Consumer to server	QLoRA (PTQ + LoRA) Frozen base + learned adapters	NF4 base + BF16 LoRA adapters	Negligible	bitsandbytes, Hugging Face
Vision / CNN Inference ResNet, EfficientNet, ViT	Any (GPU, CPU, mobile) Widely supported	PTQ Static INT8 Sym weights + asym acts	W8A8 INT8 Per-channel weights	< 0.5%	PyTorch, TensorRT, ONNX
Accuracy-Critical Tasks Medical, safety, code gen	Any (prefer H100+) More training compute	QAT (full training loop) STE + fake-quant nodes	INT8 or 4-bit Learned ranges	Minimal	PyTorch QAT, TensorRT
Sub-4-bit / Research Max compression frontier	Blackwell (B200, RTX 5090) FP4 native support	QuaRot / SpinQuant / FP4 Rotation + 2-4-bit quant	NVFP4 / INT2-3	2-5% (active R&D)	TensorRT-LLM (Blackwell)

Decision Tree: New hardware with FP8? Use FP8. | Need 4-bit GPU? Use AWQ. | Need CPU? Use GGUF. | Fine-tuning? Use QLoRA. | Best accuracy? Use QAT.

Memory Cheat Sheet (7B parameter model)

FP32: 28 GB | BF16 / FP16: 14 GB | FP8: 7 GB | INT4 / W4A16: 3.5 GB | INT2: 1.75 GB

Scale proportionally for 13B (2x), 70B (10x), 405B (58x), and add ~20% overhead for KV cache + activations during inference

Quantization in PyTorch

- PyTorch has data types corresponding to quantized tensors, which share many of the features of tensors.
- One can write kernels with quantized tensors, much like kernels for floating point tensors to customize their implementation. PyTorch supports quantized modules for common operations as part of the `torch.nn.quantized` and `torch.nn.quantized.dynamic` namespace.
- Quantization is compatible with the rest of PyTorch: quantized models are traceable and scriptable. The quantization method is virtually identical for both server and mobile backends. One can easily mix quantized and floating-point operations in a model.
- Mapping of floating-point tensors to quantized tensors is customizable with user defined observer/fake-quantization blocks. PyTorch provides default implementations that should work for most use cases.



PTQ in PyTorch

- **Post-Training Static Quantization in PyTorch**
 - Post-training static quantization involves the following steps:
 - 1. Model Preparation:** Modify the model to support quantization (insert quantization/dequantization layers).
 - 2. Calibration:** Run a calibration step with a representative dataset to gather statistics about the model (like the range of different tensors).
 - 3. Conversion:** Convert the model to use quantized weights and activations based on the calibration data.

Quantized dtypes and quantization schemes

- `torch.qscheme` — Type to describe the quantization scheme of a tensor. Supported types:
 - `torch.per_tensor_affine` — per tensor, asymmetric
 - `torch.per_channel_affine` — per channel, asymmetric
 - `torch.per_tensor_symmetric` — per tensor, symmetric
 - `torch.per_channel_symmetric` — per channel, symmetric
- `torch.dtype` — Type to describe the data. Supported types:
 - `torch.quint8` — 8-bit unsigned integer
 - `torch.qint8` — 8-bit signed integer
 - `torch.qint32` — 32-bit signed integer

<https://glaringlee.github.io/quantization.html>

PTQ Example in PyTorch (1/2)

- Step 1: Environment Setup

- First, ensure you have PyTorch installed. You might need the torchvision package for dataset and model utilities.
- Then, import the necessary libraries

```
!pip install torch torchvision
```

```
import torch
import torchvision.models as models
import torch.quantization
```

- Step 2: Load and Prepare a Model

- For this example, let's use a pre-trained model like ResNet18
- Next, we specify the quantization configuration for the model. fbgemm is one of the backends supported for x86. for ARM qnnpack is the backend to use.
- Prepare the model for quantization

```
model = models.resnet18(pretrained=True)
model.eval() # Set the model to inference mode
```

```
model.qconfig = torch.quantization.get_default_qconfig('fbgemm')
print(model.qconfig)
```

```
torch.quantization.prepare(model, inplace=True)
```

PTQ Example in PyTorch (2/2)

- Step 3: Calibration

- Now, calibrate the model with representative data. This step adjusts the internal quantization parameters (scale and zero-point) for optimal accuracy.
- Assuming you have a DataLoader `data_loader` for your dataset

```
def calibrate_model(model, data_loader):  
    model.eval()  
    with torch.no_grad():  
        for images, _ in data_loader:  
            model(images)
```

```
# Call the calibrate_model function with your data_loader  
calibrate_model(model, data_loader)
```

- Step 4: Convert to Quantized Model

- After calibration, convert the model to a quantized version

```
quantized_model = torch.quantization.convert(model, inplace=True)
```

- Step 5: Save or Use the Quantized Model

- The model is now quantized and can be saved or directly used for inference.

```
# Saving the model  
torch.save(quantized_model.state_dict(), 'quantized_model.pth')  
  
# Load and use it for inference  
quantized_model.load_state_dict(torch.load('quantized_model.pth'))  
quantized_model.eval()
```

QAT Example in PyTorch (1/2)

- Step 1: Environment Setup
 - Ensure PyTorch and torchvision are installed, then import the necessary libraries

```
import torch
import torchvision.models as models
import torch.quantization
```

- Step 2: Load and Modify the Model for QAT
 - `fuse_model` is a step specific to QAT, where we fuse certain layers (like Convolution + BatchNorm + ReLU) before quantization, as it can improve model performance.

```
model = models.resnet18(pretrained=True)
model.eval() # Set the model to inference mode

# Modify the model for QAT
model.fuse_model() # Fuses modules like Conv, BN, ReLU
model.qconfig = torch.quantization.get_default_qat_qconfig('fbgemm')

# Prepare the model for QAT
torch.quantization.prepare_qat(model, inplace=True)
```

QAT Example in PyTorch (2/2)

- Step 3: Fine-Tuning the Model with QAT →

- This involves training it for a few more epochs. During this process, the model weights and activations are simulated as quantized, which helps the model adapt to the quantization effects.

```
# Fine-tuning
# Note: You should adjust the learning rate, number of epochs, etc., based on your s
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)
criterion = torch.nn.CrossEntropyLoss()

model.train()
for epoch in range(num_epochs):
    for images, labels in data_loader: # Assuming you have a data_loader
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

- Step 4: Convert to Quantized Model →

- After fine-tuning, convert the model to a fully quantized version for inference.

```
model.eval()
quantized_model = torch.quantization.convert(model, inplace=False)
```

- Step 5: Save or Use the Quantized Model →

- The model is now quantized and can be saved or directly used for inference.

```
# Saving the model
torch.save(quantized_model.state_dict(), 'qat_quantized_model.pth')

# Load and use it for inference
quantized_model.load_state_dict(torch.load('qat_quantized_model.pth'))
quantized_model.eval()
```


Quantization Tutorial

The screenshot shows the PyTorch documentation page for Quantization. The browser address bar displays <https://docs.pytorch.org/docs/stable/quantization.html>. The page features a purple header with a survey prompt: "Help us understand how you use PyTorch! Take our quick survey. Take Survey". Below the header, the navigation bar includes the PyTorch logo, version "v2.11.0", and links for "Install PyTorch", "User Guide", "Reference API", "Developer Notes", "Community", and "Tutorials". A search bar is also present. The left sidebar, titled "Section Navigation", lists various sections under "Python API", including "torch", "Aliases in torch.nn", "torch.nn", "torch.nn.functional", "torch.Tensor", "Tensor Attributes", "Tensor Views", "torch.amp", "torch.autograd", "torch.library", "torch.accelerator", "torch.cpu", "torch.cuda", "torch.cuda.memory", "torch.mps", "torch.xpu", and "torch.mtia". The main content area is titled "Quantization" and includes the following text: "Created On: Oct 09, 2019 | Last Updated On: Dec 22, 2025". It states: "We are centralizing all quantization related development to [torchao](#), please checkout our new doc page: <https://docs.pytorch.org/ao/stable/index.html>". It then outlines migration plans for existing quantization flows: 1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode [quantize_](#) API instead; 2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx, torch.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quantization API instead (torchao.quantization.pt2e.quantize_pt2e.prepare_pt2e, torchao.quantization.pt2e.quantize_pt2e.convert_pt2e); 3. pt2e quantization has been migrated to torchao ([pytorch/ao](#)) see [pytorch/ao#2259](#) for more details. It also mentions: "We plan to delete torch.ao.quantization in 2.10 if there are no blockers, or in the earliest PyTorch version until all the blockers are cleared." Below this, the section "Quantization API Reference (Kept since APIs are still public)" is shown, followed by the text: "The [Quantization API Reference](#) contains documentation of quantization APIs, such as quantization passes, quantized tensor operations, and supported quantized modules and functions." The code snippet `torch.ao.ns.fx.utils.compute_sqnr(x, y)` is displayed, with a "[source]" link to its right.

<https://docs.pytorch.org/docs/stable/quantization.html>

References

1. [Quantizing deep convolutional networks for efficient inference: A whitepaper](#)
2. A White Paper on Neural Network Quantization (<https://arxiv.org/pdf/2106.08295.pdf>)
3. Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al., IEEE 2020]
4. Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]
5. Deep Compression [Han et al., ICLR 2016]
6. Neural Network Distiller: https://intellabs.github.io/distiller/algorithm_quantization.html
7. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]
8. BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux et al., NeurIPS 2015]
9. Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. [Courbariaux et al., Arxiv 2016]
10. XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari et al., ECCV 2016]
11. Ternary Weight Networks [Li et al., Arxiv 2016]
12. Trained Ternary Quantization [Zhu et al., ICLR 2017]

Acknowledgments:

Part of this material has been adapted from the MIT efficient Deep Learning class by Prof. Song Han, Quantization Neural Network white paper (<https://arxiv.org/pdf/2106.08295.pdf>), and Floran de Putter lecture

Attendance QR Code for April 13, 2026

